شرح كامل لكود الواجهة الأمامية (Frontend) — سطراً بسطر

عن المشروع: لوحة تحكم (Admin Dashboard) لجامعة العلوم والتقانة، مبنية بـ HTML + CSS + JavaScript فقط (بدون أي إطار عمل مثل React). الصفحة واحدة (SPA) تضم 7 شاشات، والتنقل بينها يتم بإخفاء وإظهار الأقسام. الواجهة بالعربية RTL وتدعم الوضع الليلي.

فهرس الشرح:

- أولاً: هيكل المشروع (Structure)
- الملف 1: **index.html** — هيكل الصفحة والشاشات السبع
- الملف 2: **css/style.css** — كل تنسيقات المشروع
- الملف 3: **js/mock-data.js** — البيانات التجريبية
- الملف 4: **js/main.js** — الكود المشترك بين الشاشات
- الملف 5: **js/app.js** — كود عرض كل شاشة
- الخلاصة: كيف يعمل المشروع كاملاً

أولاً: هيكل المشروع (Structure)

```
لوحة تحكم/
├── index.html          <- الصفحة الوحيدة: الهيكل + 7 شاشات داخلية
├── css/
│   └── style.css       <- ملف CSS واحد مشترك لكل الشاشات
├── js/
│   ├── mock-data.js    <- البيانات التجريبية (بديل مؤقت عن API)
│   ├── main.js         <- JavaScript مشترك (ثيم، نوافذ، تنبيهات، ترقيم)
│   └── app.js          <- دوال عرض الشاشات (جداول، بطاقات، محادثة...)
└── assets/
    ├── images/logo.png <- شعار الجامعة
    ├── icons/          <- فارغ (للمستقبل)
    └── fonts/          <- فارغ (الخطوط تُحمَّل من Google Fonts)
```

- **index.html** → الهيكل (Skeleton): سايدبار + هيدر + 7 أقسام شاشات، معظم الجداول فيه فارغة عمداً وتمتلئ بالكود.
- **css/style.css** → التصميم: الألوان، الخطوط، السايدبار، البطاقات، الجداول، النوافذ، الوضع الليلي، التجاوب، و RTL.
- **js/mock-data.js** → طبقة البيانات (Data Layer): كائن واحد فيه كل البيانات + دوال قراءة، حتى يسهل استبدالها بـ API حقيقي لاحقاً.
- **js/main.js** → الوظائف المشتركة (Shared): أي شيء تتكرر حاجته في أكثر من شاشة.
- **js/app.js** → شاشات محددة: لكل شاشة مجموعة دوال مثل renderDashboard و renderStudents.
- **/assets** → الأصول (Assets): الصور والأيقونات والخطوط المحلية.

قاعدة التنقل في المشروع: كل شاشة عنصر **div** كلاسه page-section ومعرّفه page-اسم_الشاشة. زر السايدبار يحمل سمة data-page بنفس الاسم. عند النقر تبحث دالة navigateTo عن القسم المطابق وتخفي الباقي.

الملف 1: index.html

الجزء 1: رأس الصفحة (Head)

```
<!DOCTYPE html>
<html dir="rtl" lang="ar">
<head>
  <meta charset="utf-8">
  <meta content="width=device-width, initial-scale=1.0" name="viewport">
```

- **<!DOCTYPE html>** → إعلان (Declaration) يخبر المتصفح أن الصفحة HTML5؛ يجب أن يكون أول سطر دائماً.
- **<html dir="rtl" lang="ar">** → بداية عنصر HTML الجذري.
  - **dir="rtl"** → الاتجاه من اليمين لليسار (Right-To-Left)؛ يجعل كل العناصر والنصوص تبدأ من اليمين تلقائياً لأن الواجهة عربية.
  - **lang="ar"** → لغة الصفحة عربية؛ يساعد المتصفح وقارئ الشاشة (Screen Reader) ومحركات البحث.
- **<head>** → رأس الصفحة: إعدادات لا تظهر للمستخدم.
- **<meta charset="utf-8">** → ترميز (Encoding) الحروف UTF-8؛ بدونه تظهر العربية كرموز مشوهة.
- **<meta name="viewport" ...>** → يجعل عرض الصفحة = عرض شاشة الجهاز؛ الأساس الذي يعتمد عليه التصميم المتجاوب (Responsive) على الجوال.

```
<link href="...Material+Symbols+Outlined..." rel="stylesheet">
<link href="https://fonts.googleapis.com" rel="preconnect">
<link crossorigin="" href="https://fonts.gstatic.com" rel="preconnect">
<link href="...IBM+Plex+Sans+Arabic...Cairo...JetBrains+Mono..." rel="stylesheet">
```

- **السطر الأول** → تحميل مكتبة الأيقونات (Icons) **Material Symbols** من Google؛ أي عنصر تكتب داخله "class="material-symbols-outlined واسم أيقونة يتحول لرسمة.
- **rel="preconnect"** (سطران) → "جهّز الاتصال بخوادم Google مسبقاً" فيتحمّل الخطوط أسرع.
- **crossorigin=""** → يسمح بجلب الموارد من نطاق مختلف بشكل آمن.
- **السطر الأخير** → تحميل الخطوط (Fonts): IBM Plex Sans Arabic (الأساسي للعربية)، Cairo (بديل)، JetBrains Mono (للأكواد والأرقام الجامعية). الأرقام 700;600;500;400 هي أوزان الخطوط المطلوبة. و display=swap تعني: اعرض النص فوراً بخط بديل حتى يكتمل التحميل.

```
<script src="https://cdn.tailwindcss.com"></script>
```

- **<script src="...">** → تحميل ملف JavaScript من الإنترنت (CDN = شبكة توصيل محتوى).
- **Tailwind** → إطار عمل (Framework) يولّد تنسيقات جاهزة من أسماء الكلاسات مثل flex و hidden و grid-cols-2. موجود من قبل في المشروع وليس إضافة جديدة.

```
<script id="tailwind-config">
  tailwind.config = {
    darkMode: 'class',
    theme: {
      extend: {
        'fontFamily': {
          'sans': ['"IBM Plex Sans Arabic"', '"Cairo"', '"Segoe UI"', 'sans-serif'],
          'arabic': [...], 'display': [...], 'h1': [...], 'h2': [...], 'h3': [...],
          'body': [...], 'btn': [...], 'table-h': [...], 'table-b': [...], 'badge': [...],
        },
```

- **tailwind.config = {** → إعداد Tailwind: تعريف ألوان وخطوط المشروع له.
- **darkMode: 'class'** → الوضع الليلي يُفعَّل بإضافة كلاس على عنصر HTML (إعداد داخلي لـ Tailwind).
- **theme** → إعدادات الثيم (القالب) الافتراضية.
- **extend** → "أضف فوق الافتراضي ولا تحذفه".
- **fontFamily** → تعريف عائلات الخطوط: المتصفح يجرّبها بالترتيب حتى يجد متوفراً؛ sans-serif الأخيرة أي "أي خط نظام". بقية الأسماء (arabic, btn, table-h...) كلها نفس الخطوط وأُنشئت لتسهيل تغيير خط جزء معين مستقبلاً.

```
        'colors': {
          'primary': '#091426',
          'primary-container': '#1e293b',
          'secondary': '#006591',
          'secondary-container': '#39b8fd',
          'tertiary-container': '#00301e',
          'error': '#ba1a1a',
          'error-container': '#ffdad6',
          'surface-container': '#eceef0',
          'on-primary': '#ffffff',
          'on-surface': '#191c1e',
          'on-surface-variant': '#45474c',
          ... (بقية الدرجات)
        },
```

- **colors** → لوحة ألوان المشروع بأسماء واضحة؛ كل اسم يولّد كلاسات مثل bg-primary (خلفية) و text-primary (نص) و border-primary.
- **primary #091426** → اللون الأساسي (كحلي غامق) للسايدبار والأزرار الرئيسية.
- **secondary #006591** → اللون الثانوي (أزرق) للروابط والعناصر التفاعلية.
- **tertiary** → اللون الثالث (أخضر) للنجاح والتفعيل. **error** → الأحمر للأخطاء والحذف.
- **surface** → ألوان الأسطح (خلفيات البطاقات والحقول بدرجات فاتحة متدرجة).
- **البادئة on-** → تعني "اللون المناسب للكتابة فوق كذا": on-primary أبيض يُكتب فوق الكحلي.
- **#091426** → صيغة Hex للألوان: كل زوجين يمثل كمية الأحمر/الأخضر/الأزرق من 00 إلى ff.

```
        'borderRadius': { 'DEFAULT': '0.25rem', 'lg': '0.5rem', 'xl': '0.75rem', '2xl': '1rem' },
        'spacing': { 'space-xs': '4px', 'space-sm': '8px', 'space-md': '16px', 'space-lg': '24px', 'space-xl':
'32px', 'space-2xl': '40px' }
      }
    }
  }
</script>
```

- **borderRadius** → درجات استدارة الزوايا: rounded (افتراضي 4px) حتى rounded-2xl (دائرية واضحة 16px).
- **spacing** → نظام مسافات موحد (4/8/16/24/32/40) بدل أرقام عشوائية؛ يولّد كلاسات مثل p-space-md و gap-space-sm و mb-space-lg.
- **rem** → وحدة نسبية لحجم خط الصفحة (1rem = 16px غالباً) أفضل من البكسل الثابت للإتاحة.
- **الغلق </script>** → نهاية سكربت الإعدادات.

```
<!-- التنسيق الموحد المشترك لكل الشاشات -->
<link href="css/style.css" rel="stylesheet">
<title>لوحة تحكم - إدارة قاعدة المعرفة</title>
</head>
```

- **<!-- ... -->** → تعليق (Comment) للمبرمجين؛ المتصفح يتجاهله.

- **<link rel="stylesheet"> → أهم سطر في الرأس**: ربط ملف التنسيقات الخارجي style.css. موضعه بعد سكربت Tailwind حتى تتغلب تنسيقاتنا عليه عند التعارض.
- **<title>** → النص الظاهر في تبويب المتصفح.
- **<head/>** → إغلاق الرأس.

الجزء 2: بداية الجسم + منع الوميض + حاوية التنبيهات

```
<body class="bg-surface font-body-md text-on-surface">
<script>if(localStorage.getItem('app_theme')==='dark')document.body.classList.add('dark-demo');</script>
```

- **<body>** → جسم الصفحة: كل ما يظهر للمستخدم. خلفيته فاتحة ونصه غامق بخط متوسط.
- **<script> الداخلي** → سطر واحد يُنفَّذ فوراً:
  - **localStorage** → ذاكرة تخزين دائمة في المتصفح تبقى بعد الإغلاق.
  - **getItem('app_theme')** → اقرأ القيمة المحفوظة تحت هذا المفتاح (يكتبها main.js عند تبديل الثيم).
  - **classList.add('dark-demo')** → إذا القيمة dark أضف كلاس الوضع الليلي على الجسم فوراً.
  - **السبب** → منع وميض الصفحة بيضاء ثم سوداء (مشكلة معروفة اسمها FOUC) لأن الملفات الخارجية تتأخر في التحميل.

```
<!-- Toast Container -->
<div class="fixed top-4 left-1/2 -translate-x-1/2 z-50 flex flex-col gap-2" id="toastContainer"
style="transform:translateX(-50%);"></div>
```

- **Toast** → التنبيهات المنبثقة الصغيرة (مثل "تم الحفظ بنجاح").
- **fixed** → ثابتة مكانها حتى مع التمرير. **top-4** → 16px من الأعلى.
- **left-1/2 + translateX(-50%)** → توسيط أفقي: انقلها لمنتصف الشاشة ثم ارجعها نصف عرضها للخلف.
- **z-50** → ترتيب الطبقات (z-index): فوق معظم العناصر.
- **flex flex-col gap-2** → التنبيهات المتعددة تترتب عمودياً بفراغ 8px بينها.
- **id="toastContainer"** → الاسم الذي تبحث عنه دالة showToast في main.js لإضافة الرسائل داخله. فارغ الآن ويمتلئ ديناميكياً.

الجزء 3: النافذة المشتركة (Modal)

```
<!-- Modal (نافذة واحدة مشتركة تستخدمها كل الشاشات) -->
<div class="fixed inset-0 bg-black/50 z-50 hidden items-center justify-center p-4" id="modalOverlay"
onclick="if(event.target===this)closeModal()">
```

- **Modal** → نافذة منبثقة واحدة يشاركها الجميع: إضافة طالب، تعديل خدمة، تأكيد حذف... (مبدأ Write once: اكتب مرة واستخدم في كل مكان).
- **inset-0** → تغطي الشاشة كاملة. **bg-black/50** → تعتيم أسود بشفافية 50% خلف النافذة.
- **hidden** → مخفية افتراضياً (display:none من style.css). عند الفتح تضيف main.js كلاس flex فتصير ظاهرة وموسّطة.
- **onclick="if(event.target===this)closeModal()"** → عند النقر: إذا كان الهدف event.target هو الطبقة المعتمة نفسها this (وليس محتوى النافذة) أغلقها. closeModal معرّفة في main.js.

```
  <div class="bg-white rounded-2xl shadow-lg max-w-lg w-full max-h-[90vh] overflow-y-auto">
    <div class="flex items-center justify-between p-5 border-b border-surface-container">
      <h3 class="font-h2 text-primary" id="modalTitle">العنوان</h3>
      <button class="w-8 h-8 rounded-lg hover:bg-surface-container flex items-center justify-center text-on-
surface-variant transition-all" onclick="closeModal()" aria-label="إغلاق">
        <span class="material-symbols-outlined text-xl">close</span>
      </button>
    </div>
```

- **الصندوق الأبيض** → بطاقة النافذة: زوايا دائرية كبيرة (rounded-2xl (16px، ظل shadow-lg، أقصى عرض (max-w-lg (32rem مع w-full على الجوال، وأقصى ارتفاع 90% من الشاشة [max-h-[90vh مع تمرير داخلي إذا المحتوى أطول.
- **رأس النافذة** → flex justify-between: العنوان في جهة وزر الإغلاق في المقابل، مع خط سفلي border-b يفصله عن المحتوى.
- **"id="modalTitle** → دالة openModal تغيّر نصه حسب الاستخدام ("إضافة طالب جديد"، "تأكيد الحذف"...).
- **زر الإغلاق** → 32×32 بكسل، أيقونة close (علامة X)، hover:bg-surface-container يعني تغمّق خلفيته عند مرور الفأرة، و aria-label وصف لقارئات الشاشة (إتاحة Accessibility).

```
    <div class="p-5" id="modalBody"></div>
    <div class="flex items-center justify-end gap-2 p-5 border-t border-surface-container" id="modalFooter"></div>
```

- **"id="modalBody** → جسم النافذة الفارغ؛ JavaScript يملؤه بالنماذج (حقول إدخال الطالب...).
- **"id="modalFooter** → أسفل النافذة لأزرار (إلغاء/حفظ/حذف)؛ justify-end تضعها في نهاية الاتجاه (اليسار في RTL).
- **العلاقة** → هذه العناصر الثلاثة (Title/Body/Footer) هي التي تملؤها openModal() في main.js.

الجزء 4: السايدبار (قائمة التنقل الجانبية)

```
<aside class="admin-sidebar" id="adminSidebar">
  <div class="sidebar-brand">
    <img src="assets/images/logo.png" alt="شعار الجامعة">
    <div>
      <div class="t1">جامعة العلوم والتقانة</div>
      <div class="t2">لوحة التحكم الإدارية</div>
    </div>
  </div>
```

- **<aside>** → وسم دلالي (Semantic Tag) يعني "محتوى جانبي". admin-sidebar تنسيقه الكامل في style.css (ثابت يمين الشاشة، كحلي، عرض 230px).
- **"id="adminSidebar** → يستخدمه main.js لإظهاره/إخفائه على الجوال.
- **sidebar-brand** → ترويسة الشعار: صورة 40×40 من مجلد الأصول.
- **alt="شعار الجامعة"** → نص بديل يظهر إن فشل تحميل الصورة وتقرأه قارئات الشاشة.
- **t1.** → اسم الجامعة (خط عريض 14px). **t2.** → وصف رمادي شفاف 11px — تنسيقهما في style.css.

```
  <nav class="sidebar-nav" aria-label="التنقل الرئيسي">
    <button class="side-item active" data-page="dashboard" onclick="navigateTo('dashboard')">
      <span class="material-symbols-outlined">dashboard</span>
      الرئيسية
    </button>
```

- **<nav>** → وسم دلالي لمنطقة التنقل. aria-label يصفها لقارئات الشاشة.
- **side-item** → تنسيق زر القائمة الموحد (في style.css). **active** → يميّز الصفحة الحالية وينتقل بين الأزرار ديناميكياً.
- **"data-page="dashboard** → سمة بيانات (Data Attribute) تحمل اسم الشاشة داخل العنصر؛ كود التنقل يقارن بها الزر النشط.
- **"onclick="navigateTo('dashboard')** → عند النقر تُنفَّذ دالة التنقل من app.js باسم الشاشة.
- **الأيقونة dashboard** → اسم أيقونة من مكتبة Material Symbols، وبعده نص الزر الظاهر.

```
    <button class="side-item" data-page="documents" onclick="navigateTo('documents')">... قاعدة المعرفة (folder)
</button>
    <button class="side-item" data-page="users" onclick="navigateTo('users')">... المستخدمون (group)</button>
    <button class="side-item" data-page="chatbot" onclick="navigateTo('chatbot')">... روبوت المحادثة (chat)
</button>
    <button class="side-item" data-page="services" onclick="navigateTo('services')">... الخدمات الجامعية
(business)</button>
    <button class="side-item" data-page="reports" onclick="navigateTo('reports')">... التقارير (bar_chart)
</button>
    <button class="side-item" data-page="settings" onclick="navigateTo('settings')">... الإعدادات (settings)
</button>
  </nav>
  <div class="sidebar-foot">2026 © جامعة العلوم والتقانة</div>
</aside>
```

- **كود مكرر بنمط واحد** ← الأزرار الستة الباقية مطابقة تماماً للأول؛ الاختلاف فقط في قيمة data-page واسم الأيقونة والنص.
- **<nav/>** ← إغلاق قائمة التنقل.
- **sidebar-foot** ← ذيل السايدبار: نص حقوق النشر ثابت أسفل القائمة (تنسيقه في style.css).
- **<aside/>** ← إغلاق السايدبار.

الجزء 5: الخلفية المعتمة + الهيدر

```
<!-- Backdrop (Mobile) -->
<div class="sidebar-backdrop" id="sidebarBackdrop"></div>

<!-- Main Area -->
<div class="admin-main">
```

- **sidebar-backdrop** ← طبقة معتمة تغطي الشاشة على الجوال عند فتح السايدبار؛ النقر عليها يغلق القائمة (الكود في main.js). مخفية افتراضياً وتظهر بإضافة كلاس active.
- **admin-main** ← المنطقة كلها بجانب السايدبار (هامش أيمن 230px على الشاشات الكبيرة — يصبح صفراً على الجوال عبر قاعدة @media).

```
  <header class="admin-header">
    <div class="flex items-center gap-2">
      <button class="hamburger" id="sidebarToggle" aria-label="فتح القائمة">
        <span class="material-symbols-outlined">menu</span>
      </button>
      <h1 class="font-h2 text-primary" id="pageTitle">الرئيسية</h1>
    </div>
```

- **<header>** ← وسم دلالي لرأس الصفحة. admin-header شريط ثابت (Sticky) يبقى ظاهراً أعلى الشاشة أثناء التمرير.
- **hamburger** ← زر القائمة (ثلاثة خطوط) بأيقونة menu؛ مخفي على الشاشات الكبيرة ويظهر على الجوال فقط (قاعدة @media في style.css).
- **"id="sidebarToggle** ← يستخدمه main.js لفتح السايدبار عند النقر.
- **"id="pageTitle** ← عنوان الشاشة الحالية؛ navigateTo يغيّر نصه عند كل تنقل ("الرئيسية" ← "التقارير"...).

```
    <div class="h-right">
      <button class="theme-toggle" id="themeToggle" aria-label="تبديل الوضع الليلي" title="/ الوضع الليلي
النهاري">
        <span class="material-symbols-outlined tt-moon">dark_mode</span>
        <span class="material-symbols-outlined tt-sun">light_mode</span>
      </button>
```

- **h-right** ← الجهة الأخيرة من الهيدر (يسار الشاشة في RTL) لأدوات المستخدم.

- **"theme-toggle + id="themeToggle** → زر تبديل الوضع الليلي؛ main.js يربط عليه مستمع النقر (Event Listener).
- **أيقونتان داخل زر واحد** → tt-moon (قمر) تظهر في الوضع النهاري، tt-sun (شمس) تظهر في الليلي؛ إظهارهما وإخفاؤهما بقواعد CSS: .theme-toggle .tt-sun { display:none; } وعكسها مع body.dark-demo.

```
      <div class="w-8 h-8 rounded-full bg-primary-container text-on-primary flex items-center justify-center font-bold text-xs">م</div>
      <span class="font-btn text-primary font-medium">المسؤول العام</span>
      <button class="px-3 py-2 rounded-lg bg-error text-on-primary hover:bg-error/80 font-btn transition-all" onclick="showToast('تم تسجيل الخروج بنجاح','success')">
        <span class="material-symbols-outlined text-[18px] me-1" style="vertical-align:middle;">logout</span>
        خروج
      </button>
    </div>
  </header>
```

- **الصورة الرمزية (Avatar)** → مربع 32×32 دائري بالكامل (rounded-full) بخلفية كحلية وحرف "م" موسّط.
- ** المسؤول العام** → اسم المستخدم؛ تُخفيه قاعدة @media (max-width:520px) على الشاشات الضيقة جداً.
- **زر الخروج** → أحمر bg-error بأيقونة logout؛ hover:bg-error/80 أي نفس الأحمر بشفافية 80% عند مرور الفأرة.
- **onclick="showToast(...)"** → حالياً يعرض رسالة "تم تسجيل الخروج بنجاح" فقط (رسالة تجريبية؛ الربط الفعلي بالخادم لاحقاً).
- **style="vertical-align:middle;"** → محاذاة الأيقونة عمودياً مع النص داخل الزر.
- **</header>** → إغلاق الهيدر.

الجزء 6: الشاشة 1 — الرئيسية (Dashboard)

```
  <!-- Page Content -->
  <main class="admin-content">

    <!-- ==================== DASHBOARD ==================== -->
    <div class="page-section active" id="page-dashboard">
      <p class="font-body text-on-surface-variant mb-space-lg">نظرة سريعة على حالة النظام</p>
```

- **<main>** → وسم دلالي للمحتوى الرئيسي؛ admin-content توسّطه أفقياً بعرض أقصى 1160px وحشوة 24px.
- **page-section** → **الكلاس الأهم في الملف**: كل شاشة عنصر بهذا الكلاس، وكود التنقل يظهر واحداً ويخفي البقية.
- **active** → شاشة الرئيسية وحدها نشطة عند فتح الصفحة (البقية تحمل hidden).
- **id="page-dashboard"** → navigateTo يبني الاسم هكذا: 'page-' + اسم الصفحة.
- **<p>** → سطر وصف رمادي صغير أعلى كل شاشة (نفس النمط في السبع).

```
      <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-space-md mb-space-lg">
```

- **grid** → نظام شبكة (Grid Layout) لترتيب البطاقات.
- **grid-cols-1** → عمود واحد على الجوال؛ **sm:grid-cols-2** → عمودان من الشاشة الصغيرة فأعلى؛ **lg:grid-cols-4** → أربعة أعمدة على الشاشات الكبيرة. البادئات sm:/lg: تعني "من هذا المقاس صعوداً" — هذا هو التجاوب.
- **gap-space-md** → فراغ 16px بين البطاقات. **mb-space-lg** → هامش سفلي 24px.

```
      <div class="stat-card">
        <div class="stat-ico si-teal"><span class="material-symbols-outlined">chat</span></div>
        <div>
          <div class="font-small text-on-surface-variant">محادثات اليوم</div>
          <div class="font-h1 text-primary font-bold" id="statConversations">0</div>
        </div>
      </div>
```

- **stat-card** ← بطاقة إحصائية بيضاء بإطار وظل عند مرور الفأرة (تنسيقها في style.css).
- **stat-ico** ← مربع ملوّن 42×42 للأيقونة. **si-teal** ← نسخة زرقاء؛ وتوجد si-green و si-neutral و si-gold.
- **السطر الصغير** ← وصف الإحصائية ("محادثات اليوم") بخط رمادي 13px.
- **الرقم الكبير "id="statConversations"** ← يبدأ نصاً "0" ويمتلئ بحركة عدّ عبر animateNumber() في main.js التي تبحث عن هذا المعرّف.

```
      <div class="stat-card"> ... si-green / person / مستخدمون نشطون / id="statUsers" ... </div>
      <div class="stat-card"> ... si-neutral / folder / ملفات قاعدة المعرفة / id="statDocs" ... </div>
      <div class="stat-card"> ... si-gold / check_circle / حالة النظام: يعمل بشكل طبيعي / id بدون) ... </div>
    </div>
```

- **ثلاث بطاقات مكررة بنفس البنية** ← الاختلاف فقط: اللون، الأيقونة، النص، والمعرّف.
- **بطاقة حالة النظام بدون id** ← لأن نصها ثابت لا يعدّله JavaScript.

```
    <!-- Important Alert -->
    <div id="dashboardAlert" class="mb-space-lg"></div>
```

- **حاوية فارغة** ← renderDashboardAlert() في app.js يملؤها إما بطاقة تنبيه حمراء alert-card (أسئلة بدون إجابة) أو خضراء alert-card ok (كل شيء جيد).

```
    <!-- Quick Actions -->
    <div class="grid grid-cols-1 sm:grid-cols-3 gap-space-md mb-space-lg">
      <button class="quick-action" onclick="navigateTo('documents')">
        <span class="material-symbols-outlined">upload_file</span>
        رفع مستند
      </button>
      <button class="quick-action" onclick="goToStudents()">... إضافة طالب (person_add)</button>
      <button class="quick-action" onclick="navigateTo('chatbot')">... اختبار الروبوت (chat)</button>
    </div>
```

- **quick-action** ← زر إجراء سريع بشكل بطاقة (تنسيقه في style.css): أيقونة زرقاء + نص، وترتفع قليلاً مع ظل عند hover.
- **الأزرار الثلاثة** ← تختصر الوصول لأكثر مهام: navigateTo('documents') لقاعدة المعرفة، goToStudents() (دالة تفتح شاشة المستخدمين وتنشّط تبويب الطلاب)، navigateTo('chatbot') للروبوت.

```
    <!-- Recent Activity -->
    <div class="panel-card">
      <div class="flex items-center justify-between mb-space-sm">
        <h3 class="font-h3 text-primary">آخر الأنشطة</h3>
        <button class="utab" style="margin-left:0;" onclick="navigateTo('reports')">عرض السجل الكامل</button>
      </div>
      <div id="recentActivityList"></div>
    </div>
  </div>
```

- **panel-card** ← اللوحة البيضاء العامة — أبسط مكون مشترك، كل الشاشات تستخدمه.

- **رأس اللوحة** ← عنوان يمين وزر يسار (justify-between).
- **utab** ← نفس تنسيق تبويبات المستخدمين لكنه هنا زر نصي ينقل للتقارير؛ "style="margin-left:0;" يتغلب على هامشه الافتراضي لأنه وحده في السطر.
- **"id="recentActivityList** ← فارغ؛ renderRecentActivity() يملؤه بآخر 5 أنشطة كسطور recent-row.
- ** الأخير** ← إغلاق قسم شاشة الرئيسية.

الجزء 7: الشاشة 2 — قاعدة المعرفة (Documents)

```
    <div class="page-section hidden" id="page-documents">
      <p class="font-body text-on-surface-variant mb-space-lg">رفع وإدارة ملفات المساعد الذكي</p>
```

- **hidden** ← مخفية افتراضياً؛ تظهر فقط عند النقر على زرها في السايدبار (كلاس hidden معرّف في style.css كـ display:none).

```
      <div class="panel-card mb-space-lg">
        <h3 class="font-h3 text-primary mb-space-md">رفع مستند جديد</h3>
        <div class="drop-zone border-2 border-dashed border-surface-container rounded-xl p-space-lg text-center
cursor-pointer hover:border-secondary transition-all"
             ondragover="handleDragOver(event)" ondragleave="handleDragLeave(event)" ondrop="handleDrop(event)"
onclick="document.getElementById('fileInput').click()">
```

- **drop-zone** ← منطقة سحب وإفلات (Drag & Drop) الملفات: إطار متقطع border-dashed بسماكة 2px، زوايا دائرية، ومؤشر يد cursor-pointer.
- **ondragover** ← أثناء سحب ملف فوقها: handleDragOver يضيف كلاس border-secondary فيتلوّن الإطار أزرق.
- **ondragleave** ← عند خروج الملف: handleDragLeave يزيل الكلاس فيعود الرمادي.
- **ondrop** ← عند إفلات الملف: handleDrop يقرأ اسمه ويبدأ شريط التقدم.
- **"onclick="document.getElementById('fileInput').click()"** ← النقر على المنطقة يستدعي برمجياً حدث النقر على حقل الملفات المخفي بالأسفل فيفتح نافذة الاختيار. كل هذه الدوال الأربع في app.js.

```
          <span class="material-symbols-outlined text-[48px] text-outline-variant mx-auto mb-space-
sm">cloud_upload</span>
          <h3 class="font-h3 text-primary mb-space-sm">اسحب الملفات هنا أو اضغط للرفع</h3>
          <p class="font-body text-on-surface-variant">يدعم PDF, DOCX, PPTX, XLSX, ZIP - الحد الأقصى 200
ميجابايت</p>
        </div>
        <input type="file" id="fileInput" multiple style="display:none" onchange="handleFileSelect(event)">
```

- **محتوى المنطقة** ← أيقونة سحابة كبيرة (48px) موسّطة mx-auto + عنوان + نص الأنواع المدعومة.
- **"input type="file"** ← حقل اختيار ملفات مخفي عمداً (display:none) لأن شكله الافتراضي غير جميل.
- **multiple** ← يسمح باختيار أكثر من ملف مرة واحدة.
- **"onchange="handleFileSelect(event)** ← عند اختيار ملف من النافذة تنطلق نفس آلية الرفع المستخدمة مع السحب والإفلات.

```
        <div id="uploadProgress" class="hidden mt-space-md">
          <div class="flex items-center gap-2 p-3 rounded-lg bg-surface-container-low">
            <div class="w-8 h-8 rounded-lg bg-secondary-container text-on-secondary-container flex items-center
justify-center">
              <span class="material-symbols-outlined text-[18px]">upload</span>
            </div>
            <div class="flex-1">
              <div class="font-btn text-primary" id="uploadFileName">ملف...</div>
              <div class="w-full bg-surface-container-high rounded-full h-1.5 overflow-hidden mt-1">
                <div class="bg-secondary h-1.5 rounded-full transition-all" id="uploadProgressFill"
style="width:0%"></div>
              </div>
            </div>
            <span class="font-code-sm text-secondary font-semibold" id="uploadPercent">0%</span>
          </div>
        </div>
      </div>
```

- **شريط التقدم (Progress Bar)** → مخفي ويتحكم به simulateUpload() في app.js.
- **"id="uploadProgress** → الحاوية: تُزال منها hidden عند بدء الرفع وتُعاد بعد الانتهاء.
- **"id="uploadFileName** → اسم الملف المرفوع.
- **المسار الرمادي** → ارتفاعه h-1.5 (6px) ودائري الأطراف؛ داخله الشريط الأزرق **"id="uploadProgressFill** الذي يتحرك عرضه من 0% إلى 100% (style="width:0%" هي القيمة الابتدائية التي يعدّلها الكود).
- **"id="uploadPercent** → نص النسبة المئوية بجانب الشريط بخط الأكواد.
- **flex-1** → العمود الأوسط يتمدد ليأخذ المساحة المتبقية بين الأيقونة والنسبة.

```
      <!-- Files Table -->
      <div class="panel-card">
        <div class="flex flex-col md:flex-row items-center gap-space-sm mb-space-md">
          <div class="relative flex-1 w-full">
            <span class="material-symbols-outlined absolute start-3.5 top-1/2 text-outline text-[20px]"
style="transform:translateY(-50%);">search</span>
            <input class="w-full h-11 ps-11 pe-10 rounded-lg bg-surface-container-low text-on-surface font-body
placeholder:text-outline focus:outline-none transition-all" id="docSearch" placeholder="البحث باسم المستند..."
type="text" oninput="filterDocuments()">
          </div>
          <select class="h-11 px-3 rounded-lg bg-surface-container-low text-on-surface font-btn focus:outline-none
cursor-pointer" id="docCategory" onchange="filterDocuments()">
            <option value="">جميع التصنيفات</option>
          </select>
        </div>
```

- **شريط البحث والفلترة** → الحاوية flex تكون عمودية flex-col على الجوال وأفقية md:flex-row من الشاشة المتوسطة.
- **أيقونة داخل حقل البحث** → الحاوية relative والأيقونة absolute: أي الأيقونة موضوعة داخل حدود الحقل لا فوقه. start-3.5 أي 14px من بداية الاتجاه (اليمين في RTL)، و top-1/2 + translateY(-50%) توسّطها عمودياً.
- **ps-11 / pe-10** → حشوة بداية 44px (ترك مكان للأيقونة) ونهاية 40px.
- **id="docSearch" + oninput** → كل ضغطة حرف تستدعي filterDocuments() التي تعيد رسم الجدول بالنتائج المطابقة فوراً.
- **placeholder** → النص الرمادي الإرشادي داخل الحقل عند فراغه.
- **"id="docCategory** → قائمة تصنيفات فيها خيار واحد في HTML فقط؛ الباقي تضيفه renderDocuments() تلقائياً من البيانات.

```
        <div class="overflow-x-auto">
          <table class="w-full text-start text-on-surface font-table-b border-collapse">
            <thead>
              <tr class="bg-surface-container text-on-surface-variant font-table-h">
                <th class="py-3 px-3 text-start">المستند</th><th ...>النوع والحجم</th><th ...>التاريخ</th><th
...>الحالة</th><th ... text-end>إجراء</th>
              </tr>
            </thead>
            <tbody id="documentsBody" class="divide-y"></tbody>
          </table>
        </div>
        <div class="flex items-center justify-center gap-1 mt-space-md" id="docPagination"></div>
      </div>
    </div>
```

- **overflow-x-auto** ← إذا الجدول أعرض من الشاشة يظهر تمرير أفقي بدل كسر التصميم.
- **<table>** ← جدول بعرض كامل؛ border-collapse تدمج حدود الخلايا في خط واحد.
- **<thead>** ← رأس الجدول: صف رمادي bg-surface-container بخمسة أعمدة th. text-start للبيانات و text-end لعمود الإجراءات (أطراف الصف في RTL).
- **<tbody id="documentsBody" class="divide-y">** ← **فارغ تماماً**: renderDocuments() يرسم الصفوف داخلها. divide-y ترسم خطاً فاصلاً بين الصفوف.
- **id="docPagination"** ← حاوية أزرار الصفحات (1، 2، 3...) يرسمها renderPagination() المشترك في main.js.

الجزء 8: الشاشة 3 — المستخدمون (طلاب + حسابات وصلاحيات)

```
    <div class="page-section hidden" id="page-users">
      ...
      <div class="flex border-b border-surface-container mb-space-lg">
        <button class="utab active" id="tabStudents" onclick="switchUserTab('students')">الطلاب</button>
        <button class="utab" id="tabAccounts" onclick="switchUserTab('accounts')">حسابات الإدارة
والصلاحيات</button>
      </div>
```

- **شاشة بتبويبين داخليين (Tabs)** ← تبويب الطلاب نشط افتراضياً (كلاس active عليه).
- **switchUserTab(...)** ← دالة في app.js تُظهر لوحة وتخفي الأخرى وتنقل كلاس active بين الزرين.
- **border-b** ← خط سفلي للحاوية يعمل كمسار يتحرك عليه مؤشر التبويب النشط.

```
      <div id="panelStudents">
        <div class="panel-card">
          ... أيقونة داخل حقل :نفس نمط بحث المستندات + input id="studentSearch" oninput="filterStudents()" ...
          <button ... onclick="openAddStudentModal()">
            <span class="material-symbols-outlined text-[20px]">person_add</span>
            إضافة طالب
          </button>
          <table>
            <thead> إجراء | الدفعة | الكلية | الاسم | الرقم الجامعي </thead>
            <tbody id="studentsBody" class="divide-y"></tbody>
          </table>
          <div ... id="studentPagination"></div>
        </div>
      </div>
```

- **panelStudents** ← لوحة الطلاب (ظاهرة افتراضياً). **كود البحث مكرر بنفس نمط شاشة المستندات** — تكرار مقصود لتوحيد الشكل.

- **زر إضافة طالب** → يفتح النافذة المشتركة عبر openAddStudentModal() من app.js.
- **"id="studentsBody** → جسم الجدول الفارغ؛ يملؤه renderStudents() بصفوف الطلاب. و "id="studentPagination لأزرار الترقيم.

```
  <div id="panelAccounts" class="hidden">
    <div class="panel-card mb-space-lg">
      <h3 ...>الحسابات</h3>
      <table>
        <thead> تفعيل | الحالة | الدور | الاسم </thead>
        <tbody id="usersBody" class="divide-y"></tbody>
      </table>
    </div>
    <div class="panel-card">
      <h3 ...>مصفوفة الصلاحيات</h3>
      <table class="... min-w-[700px]" id="permissionsMatrix">
        <thead id="permHead"></thead>
        <tbody id="permBody"></tbody>
      </table>
    </div>
  </div>
</div>
```

- **panelAccounts** → لوحة الحسابات مخفية افتراضياً وتظهر عند اختيار التبويب الثاني.
- **"id="usersBody** → يملؤه renderUsers() بصفوف الحسابات مع مفاتيح تفعيل (toggle-switch) لكل حساب.
- **مصفوفة الصلاحيات** → جدول رأسه وجسمه فارغان (permHead/permBody) لأن عدد الأعمدة يتغير حسب الصلاحيات؛ renderPermissions() يرسمهما كاملَيْن. min-w-[700px] حد أدنى للعرض مع تمرير أفقي حتى لا تنضغط الأعمدة الثمانية.

الجزء 9: الشاشة 4 — روبوت المحادثة (Chatbot)

```
<div class="page-section hidden" id="page-chatbot">
  ...
  <div class="grid grid-cols-1 lg:grid-cols-2 gap-space-lg" style="height:calc(100vh - 230px);">
```

- **عمودان** → الإعدادات يميناً والمحادثة يساراً على الشاشات الكبيرة، وعمود واحد فوق بعض على الجوال.
- **"style="height:calc(100vh - 230px)** → ارتفاع الشاشة كاملة ناقص 230px (مساحة الهيدر والعنوان)؛ calc عملية حسابية داخل CSS و vh وحدة = 1% من ارتفاع الشاشة.

```
        <div class="panel-card overflow-y-auto">
          <h3 ...>إعدادات البوت</h3>
          <div class="space-y-3">
            <div>
              <label class="font-label-sm text-on-surface-variant font-semibold">اسم البوت</label>
              <input class="form-input w-full h-11 px-3 rounded-lg bg-surface-container-low text-on-surface font-
body focus:outline-none transition-all" id="cbBotName" value="مساعد الجامعة">
            </div>
            <div><label ...>رسالة الترحيب</label>
              <textarea class="form-textarea w-full p-3 ..." id="cbWelcome" rows="3">مرحباً بك في مساعد
الجامعة!...</textarea></div>
            <div class="grid grid-cols-2 gap-space-sm">
              <div><label>اللهجة</label><select class="form-select ..." id="cbDialect">عربي|خليجي|مصري|سعودي
فصحى</select></div>
              <div><label>الرد الافتراضي</label><input ... id="cbDefault" ...></div>
            </div>
            <div class="grid grid-cols-2 gap-space-sm">
              <div><label>Temperature</label><input ... id="cbTemp" value="0.7" step="0.1"></div>
              <div><label>Top-K</label><input ... id="cbTopK" value="50"></div>
            </div>
            <div><label>System Prompt</label><textarea ... id="cbPrompt" rows="3">أنت مساعد جامعي ذكي...
</textarea></div>
            <button class="w-full ... bg-primary ..." onclick="saveChatbotSettings()">حفظ الإعدادات</button>
          </div>
        </div>
```

- **نموذج (Form) إعدادات الروبوت** → نمط موحد: label وصف صغير فوق كل حقل إدخال.
- **form-input / form-textarea / form-select** → كلاسات تنسيق موحدة للحقول في style.css، تشمل لون الإطار الأزرق عند التركيز (focus).
- **value="..."** → القيمة الابتدائية الظاهرة داخل الحقل. **rows="3"** → ارتفاع مربع النص 3 أسطر.
- **grid-cols-2** → الحقول المرتبطة تظهر جنب بعض (اللهجة + الرد الافتراضي، ثم Temperature + Top-K).
- **step="0.1"** → مقدار الزيادة عند استخدام أسهم الحقل الرقمي.
- **الحقول معرّفة لكن غير مرتبطة بالكود حالياً** → زر الحفظ يعرض رسالة نجاح فقط (نسخة تجريبية واجهة أمامية).

```
        <div class="panel-card flex flex-col" style="height:100%;">
          <div class="flex items-center gap-2 py-space-sm border-b border-surface-container">
            <span class="font-h3 text-primary" id="chatBotNameDisplay">جامعة العلوم والتقانة</span>
          </div>
          <div class="flex-1 overflow-y-auto py-space-md space-y-2" id="chatMessages"></div>
          <div class="flex items-center gap-2 py-space-sm border-t border-surface-container">
            <input class="flex-1 h-11 px-3 ..." id="chatInput" placeholder="اكتب رسالتك..."
onkeypress="if(event.key==='Enter')sendChat()">
            <button class="px-4 py-2 ... bg-primary ..." onclick="sendChat()" aria-label="إرسال">
              <span class="material-symbols-outlined text-[20px]">send</span>
            </button>
          </div>
        </div>
      </div>
    </div>
```

- **نافذة اختبار المحادثة** → بنية ثلاثية عمودية (flex flex-col): رأس + رسائل + إدخال، مثل تطبيقات الدردشة.
- **id="chatBotNameDisplay"** → اسم البوت؛ يحدثه initChatMessages() من الإعدادات.
- **id="chatMessages"** → منطقة الرسائل: flex-1 تأخذ كل المساحة المتبقية، و overflow-y-auto تمرير عند الامتلاء، و space-y-2 فراغ 8px بين الرسائل. فارغة ويمتلئ ديناميكياً.
- **id="chatInput" + onkeypress** → حقل الكتابة؛ الضغط على Enter (event.key==='Enter') يرسل الرسالة بدون لمس الزر.

- **زر الإرسال** → أيقونة send يستدعي sendChat().

الجزء 10: الشاشة 5 — الخدمات الجامعية (Services)

```
    <div class="page-section hidden" id="page-services">
      <div class="panel-card mb-space-lg">
        <h3 ...>معلومات الجامعة</h3>
        <p ...>2020 جامعة العلوم والتقانة - تأسست عام...</p>
        <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-space-sm">
          <div class="p-3 rounded-lg bg-surface-container-low">
            <div class="font-caption text-on-surface-variant mb-1">الاسم</div>
            <div class="font-body-md text-on-surface">جامعة العلوم والتقانة</div>
          </div>
          ... (الموقع / البريد الإلكتروني / الهاتف بنفس البنية) ...
        </div>
      </div>
      <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-space-lg" id="servicesGrid"></div>
    </div>
```

- **بطاقة معلومات الجامعة** → ثابتة في HTML: أربعة صناديق صغيرة (اسم/موقع/بريد/هاتف) كل صندوق: وصف رمادي صغير فوق قيمة أوضح.
- **تحول الشبكة** → عمود واحد على الجوال ← عمودان ← 4 أعمدة على الكبيرة.
- **"id="servicesGrid** → شبكة فارغة؛ renderServices() يرسم داخلها بطاقة لكل خدمة من البيانات (الشؤون المالية، الرسوم، جول الامتحانات) بشكل panel-card.

الجزء 11: الشاشة 6 — التقارير (Reports)

```
    <div class="page-section hidden" id="page-reports">
      <div class="panel-card mb-space-lg">
        <div class="flex items-center justify-between mb-space-md">
          <h3 ...>استخدام البوت</h3>
          <div class="flex gap-space-xs">
            <button class="px-3 py-1 rounded-lg bg-primary text-on-primary font-btn" id="chartWeekBtn"
onclick="switchChartPeriod('week')">أسبوع</button>
            <button class="px-3 py-1 rounded-lg bg-surface-container text-on-surface-variant font-btn"
id="chartMonthBtn" onclick="switchChartPeriod('month')">شهر</button>
          </div>
        </div>
        <div style="height:260px;position:relative;">
          <canvas id="usageChart"></canvas>
        </div>
      </div>
```

- **قسم الرسم البياني** → زرا اختيار المدة: "أسبوع" نشط (bg-primary كحلي) و"شهر" غير نشط (رمادي).
- **switchChartPeriod(...)** → تبدل البيانات وتنقل التنسيق النشط بين الزرين (chartWeekBtn/chartMonthBtn).
- **<"canvas id="usageChart** → لوحة رسم (Canvas) ترسم عليها مكتبة Chart.js الرسم الخطي لمعدل الاستخدام.
- **الحاوية height:260px + position:relative** → شرط تقني من Chart.js حتى يعمل الرسم متجاوباً داخل مساحة محددة.

```
      <div class="grid grid-cols-1 lg:grid-cols-2 gap-space-lg mb-space-lg">
        <div class="panel-card">
          <h3 ...>أكثر الأسئلة تكراراً</h3>
          <table><thead> # | التكرار | السؤال </thead><tbody id="frequentQuestionsBody" class="divide-y"></tbody>
</table>
        </div>
        <div class="panel-card">
          <div class="flex items-center justify-between mb-space-md">
            <h3 ...>الأسئلة الفاشلة</h3>
            <span class="px-2.5 py-0.5 rounded-full font-badge bg-error-container text-error font-semibold">تحتاج
انتباه</span>
          </div>
          <table><thead> إجراء | حالة الردود | السؤال </thead><tbody id="failedQuestionsBody" class="divide-y">
</tbody></table>
        </div>
      </div>
```

- **جدولان جنباً إلى جنب** ← عمودان على الشاشات الكبيرة وعمودان متراكبان على الصغيرة.
- **"id="frequentQuestionsBody** ← يملؤه renderReportTables() بأكثر الأسئلة تكراراً.
- **الشارة (Badge)** ← كبسولة حمراء صغيرة "تحتاج انتباه": rounded-full تجعلها دائرية الأطراف و bg-error-container خلفية وردية و text-error نص أحمر.
- **"id="failedQuestionsBody** ← نفس الدالة تملؤه بالأسئلة بدون ردود وزر مراجعة لكل سؤال.

```
      <div class="panel-card">
        <div class="flex flex-col md:flex-row items-center gap-space-sm mb-space-md">
          <input type="date" class="h-11 px-3 ..." id="activityDateFilter" onchange="filterActivities()">
          <select class="h-11 px-3 ..." id="activityUserFilter" onchange="filterActivities()">
            <option value="">كل المستخدمين</option>
          </select>
          <button class="... bg-surface-container-high hover:bg-surface-container-highest ... md:ms-auto"
onclick="exportCSV()">
            <span class="material-symbols-outlined text-[18px]">download</span>
            تصدير CSV
          </button>
        </div>
        <table><thead> الوقت | الإجراء | المستخدم | IP </thead><tbody id="activityBody" class="divide-y"></tbody>
</table>
        <div ... id="activityPagination"></div>
      </div>
    </div>
```

- **سجل النشاط** ← جدول بأربعة أعمدة: المستخدم، الإجراء، الوقت، IP.
- **"id="activityDateFilter** ← فلتر تاريخ (input type=date) يخفي الصفوف غير المطابقة عند التغيير عبر filterActivities().
- **"id="activityUserFilter** ← قائمة المستخدمين؛ خيار واحد فقط في HTML والباقي تضيفه renderActivities() من البيانات.
- **md:ms-auto** ← يدفع زر التصدير لأقصى يسار الصف من الشاشة المتوسطة فأعلى (هامش بداية تلقائي).
- **()exportCSV** ← ينزّل السجل كاملاً كملف CSV يفتح في Excel.
- **"id="activityBody" + id="activityPagination** ← الجدول الفارغ وترقيمه.

الجزء 12: الشاشة 7 — الإعدادات (Settings)

```
    <div class="page-section hidden" id="page-settings">
      <div class="panel-card max-w-2xl">
        <div class="space-y-0">
          <div class="flex items-center justify-between p-space-md border-b border-surface-container gap-3 flex-
wrap">
            <div>
              <h4 class="font-body-md text-primary font-semibold">اسم الجامعة</h4>
              <p class="font-small text-on-surface-variant">اسم الجامعة كما يظهر في النظام</p>
            </div>
            <input class="form-input max-w-xs h-11 px-3 ..." id="setUniName">
          </div>
          ... (اللغة / رفع بزر الجامعة شعار :مشابهة صفوف select id="setLanguage" / مفتاح API id="setApiKey"
type="password" الجلسة مدة / إظهار زر مع id="setSession" / الدخول محاولات id="setAttempts") ...
          <div class="flex items-center justify-end p-space-md pt-space-lg">
            <button ... onclick="saveSettings()">حفظ الإعدادات</button>
          </div>
        </div>
      </div>
    </div>
```

- **شاشة الإعدادات** → قائمة صفوف (Setting Item): كل صف وصفه يميناً وحقل إدخاله يساراً وخط سفلي يفصل الصفوف.
- **max-w-2xl** → عرض أقصى 42rem حتى لا تتمدد الصفوف على الشاشات العريضة. flex-wrap تسمح للصف بالالتفاف على الشاشات الضيقة.
- **معرّفات الحقول** → (setUniName, setLanguage, setApiKey, setSession, setAttempts) هي التي يملؤها renderSettings() من mock-data.js عند فتح الصفحة.
- **type="password"** → يخفي مفتاح API بنقاط؛ وزر "إظهار" يستدعي toggleApiKey() التي تحوله إلى text فتظهر المحارف.
- **زر الحفظ** → saveSettings() تعرض رسالة نجاح فقط (تجريبي).

الجزء 13: نهاية الصفحة + ترتيب تحميل السكربتات

```
  </main>
</div>

<!-- ترتيب التحميل: البيانات ← الكود المشترك ← شاشات اللوحة -->
<script src="js/mock-data.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
<script src="js/main.js"></script>
<script src="js/app.js"></script>
</body>
</html>
```

- **</main> ثم ** → إغلاق منطقة المحتوى ثم إغلاق admin-main.
- **ترتيب السكربتات مهم جداً** (يوضحه التعليق):
  - 1) mock-data.js أولاً → لأن كل الدوال الأخرى تقرأ البيانات منه.
  - 2) مكتبة Chart.js من CDN → يحتاجها رسم التقارير.
  - 3) main.js → المشترك (ثيم/سايدبار/نوافذ/تنبيهات).
  - 4) app.js أخيراً → لأنه يستدعي دوال main.js ويقرأ البيانات.
- **لماذا في نهاية body وليس head؟** → حتى يُبنى هيكل الصفحة أولاً ثم تُنفَّذ الأكواد؛ فلا يبحث الكود عن عناصر لم تُنشأ بعد.
- **</body> + </html>** → إغلاق الجسم وإغلاق المستند.

ملخص ملف index.html

- **وظيفة الملف:** الهيكل الكامل للوحة التحكم — سايدبار تنقل + هيدر + 7 شاشات تظهر منها واحدة فقط في كل لحظة.
- **أهم ما فيه:** ربط Tailwind وإعداداته، ربط style.css، ثلاثة مكونات مشتركة (Toast، Modal واحد، السايدبار/الهيدر)، وكل شاشة div.page- section بمعرّف page-اسم.
- **علاقته بالملفات الأخرى:** يربط style.css للتنسيق، ويحمّل السكربتات بالترتيب، ويستدعي دوالها مباشرة عبر onclick.
- **أهم فكرة:** HTML هو "الهيكل الفارغ" و JavaScript هو "من يملؤه بالبيانات" — علاقة الربط عبر id و data-page.

الملف 2: css/style.css

هذا الملف هو **التنسيق الموحد المشترك لكل الشاشات**. منظم بفهرس من 9 أقسام مرقمة، ويُقرأ من الأعلى للأسفل. المبادئ المستخدمة فيه: متغيرات CSS للألوان، كلاسات قابلة لإعادة الاستخدام، وقسم كامل للوضع الليلي.

القسم 0: التعليق الافتتاحي والفهرس

```
/* ============================================================
   لوحة تحكم الجامعة – css/style.css
   الملف الوحيد المشترك لكل الشاشات (RTL + وضع ليلي)
   ------------------------------------------------------------
   الفهرس:
   01) المتغيرات (Colors / Fonts / Radius)
   02) الأساسيات (Base)
   03) الخطوط (Typography)
   04) الحركات (Animations)
   05) التخطيط: السايدبار + الهيدر + المحتوى
   06) المكونات: بطاقات، تنبيهات، تبويبات، نماذج، مفاتيح...
   07) أدوات مساعدة (Utilities)
   08) الوضع الليلي (Dark Mode)
   09) التجاوب مع الشاشات (Responsive)
   ============================================================ */
```

- **/* ... */** → تعليق (Comment) CSS؛ المتصفح يتجاهله. الفهرس يساعد أي مطور على الوصول للقسم المطلوب بسرعة.

القسم 01: المتغيرات (CSS Variables)

```
:root {
  --font-ar: 'IBM Plex Sans Arabic', 'Cairo', 'Tajawal', 'Segoe UI', sans-serif;
  --font-mono: 'JetBrains Mono', monospace;
```

- **:root** → محدد (Selector) يعني "عنصر html الجذر"؛ المتغيرات المعرفة هنا متاحة في كل الصفحة.
- **--font-ar** → متغير (Variable) يحمل قائمة الخطوط العربية بالترتيب؛ بدل تكرار القائمة 15 مرة في الملف نكتب var(--font-ar). المتصفح يجرّب الخطوط يساراً ليميناً حتى يجد متوفراً.
- **--font-mono** → خط أحادي المسافة (كل الحروف بعرض متساوٍ) للأكواد والأرقام.

```
  /* الألوان الأساسية */
  --c-primary: #091426;
  --c-primary-container: #1e293b;
  --c-on-primary: #ffffff;
  --c-secondary: #006591;
  --c-secondary-hover: #005577;
  --c-secondary-container: #39b8fd;
  --c-on-secondary-container: #004666;
  --c-secondary-fixed: #c9e6ff;
  --c-secondary-fixed-variant: #89ceff;
  --c-success: #00a472;
  --c-success-container: #00301e;
  --c-success-fixed: #6ffbbe;
  --c-error: #ba1a1a;
  --c-error-container: #ffdad6;
```

- **--c-primary** → الكحلي الغامق: خلفية السايدبار والنصوص الرئيسية والأزرار الأساسية.
- **--c-primary-container** → كحلي أفتح قليلاً (خلفية الأفاتار وحالة hover للأزرار الكحلية).
- **--c-on-primary** → أبيض: لون النص فوق الكحلي. البادئة on- تعني "فوق".
- **--c-secondary** → الأزرق الأساسي (روابط، أيقونات، إطار التركيز) و **--c-secondary-hover** درجته عند مرور الفأرة.
- **--c-secondary-container** → أزرق سماوي فاتح (أيقونة الرفع، شريط التقدم، رسائل المحادثة) و **--c-on-secondary-container** النص الأزرق الغامق فوقه.
- **--c-secondary-fixed / variant** → درجات زرقاء فاتحة للشارات (Badges) والنصوص في التقارير.
- **--c-success** → الأخضر: مفاتيح التفعيل والحالة "نشط" وحدود التنبيه السليم.
- **--c-success-container** → أخضر داكن جداً (خلفية الشارات الخضراء مع نص فاتح).
- **--c-error** → الأحمر: زر الخروج والحذف والتنبيهات. **--c-error-container** → وردي فاتح كخلفية شارات الخطأ.

```
  /* لون مميز (الأنشطة والتبويبات النشطة) */
  --c-accent: #3a6db5;
  --c-accent-light: #a3c0ee;
  --c-accent-bg: #e8eefc;
```

- **--c-accent** → أزرق مميز: نقطة النشاط الأخير وخط التبويب النشط ولون أيقونة si-gold.
- **--c-accent-light** → نسخة فاتحة منه لنص الزر النشط في السايدبار.
- **--c-accent-bg** → خلفية زرقاء شاحبة جداً.

```
  /* الأسطح والحدود */
  --c-surface-low: #f2f4f6;
  --c-surface-container: #eceef0;
  --c-surface-high: #e6e8ea;
  --c-surface-highest: #e0e3e5;
  --c-border: #e6e8ea;
  --c-border-light: #eef0f3;
  --c-outline: #75777d;
  --c-outline-variant: #c5c6cd;
```

- **سلم الأسطح (Surface Tones)** → 4 درجات رمادية فاتحة من الأفتح للأغمق: خلفيات حقول البحث، رؤوس الجداول، خلفيات hover... تدرّجها موحد في كل المشروع.
- **--c-border** → رمادي فاتح لحدود البطاقات والجداول وخطوط الفصل.
- **--c-outline** → رمادي متوسط للأوصاف والنصوص الثانوية، و **--c-outline-variant** رمادي فاتح لأيقونات كبيرة شاحبة (سحابة الرفع).

```
  /* النصوص */
  --c-text: #191c1e;
  --c-text-secondary: #45474c;
```

- **--c-text** → أسود مزرق غامق: النص الأساسي.
- **--c-text-secondary** → رمادي غامق: النصوص الثانوية والأيقونات في الجداول.

```
  /* زوايا وظلال */
  --radius-sm: 0.5rem;
  --radius-md: 0.75rem;
  --radius-lg: 1rem;
  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
  --tr: all 0.2s ease;
}
```

- **--radius-sm/md/lg** → ثلاث درجات استدارة (8/12/16px) موحدة بدل أرقام متفرقة.
- **--shadow-sm / --shadow-lg** → ظلان جاهزان. صيغة الظل: إزاحة أفقية ثم رأسية ثم تشويش (Blur) ثم انتشار ثم لون بشفافية 5% أو 10%.
- **--tr** → اختصار انتقال (Transition): "طبّق حركة ناعمة 0.2 ثانية على كل الخصائص" — يعاد استخدامه في كل الأزرار.
- **}** → إغلاق كتلة :root.

القسم 02: الأساسيات (Base)

```
@layer base {
  html, body { margin: 0; padding: 0; font-family: var(--font-ar); }
  body { overscroll-behavior: none; }
  main > :first-child { margin-top: 0 !important; }
  main > :last-child { margin-bottom: 0 !important; }
}
```

- **@layer base** → طبقة (Layer) تنسيقية: يضعها Tailwind للأنماط الأساسية؛ وضع قواعدنا داخلها يحافظ على نفس ترتيب الأولوية الذي كان في المشروع.
- **html, body { margin:0; padding:0; }** → تصفير الهوامش الافتراضية للمتصفح (Reset) حتى لا تظهر فراغات بيضاء حول الصفحة، مع تطبيق الخط العربي.
- **overscroll-behavior: none** → منع حركة "الارتداد" (Bounce) عند نهاية التمرير على الجوال.
- **main > :first-child** → أول عنصر داخل main بلا هامش علوي (علامة !important تعني "تغلب على أي قاعدة أخرى")، والعكس للعنصر الأخير — حتى يلتصق المحتوى بالهيدر والنهاية بدقة.

```
html, body { overflow-x: hidden; overflow-x: clip; }
::-webkit-scrollbar { display: none; }
::selection { background: rgba(0, 101, 145, 0.18); }
```

- **overflow-x** → منع التمرير الأفقي نهائياً (قيمتان مكررتان عمداً: clip أحدث ويدعمها متصفحان أحدث و hidden احتياط).
- **::-webkit-scrollbar { display:none }** → إخفاء شريط التمرير تماماً في متصفحات كروم (شكل أنظف؛ التمرير بالعجلة يعمل عادي).
- **::selection** → لون التظليل عند تحديد نص بالماوس: أزرق شفاف 18% بدل الأزرق الافتراضي للمتصفح.

```
button:focus-visible, input:focus-visible, select:focus-visible,
textarea:focus-visible, a:focus-visible {
  outline: 2px solid var(--c-secondary); outline-offset: 2px; border-radius: var(--radius-sm);
}
```

- **:focus-visible** → حالة "التركيز بلوحة المفاتيح" (Tab) فقط، لا تظهر مع نقر الماوس — إتاحة (Accessibility) لمن يتنقل بالكيبورد.
- **outline** → إطار أزرق بسماكة 2px حول العنصر المركّز، و outline-offset يبعده 2px عن الحدود، وزواياه دائرية.

```
button { transition: var(--tr); }
button:not(:disabled):active { transform: scale(0.98); }
```

- **transition: var(--tr)** → كل الأزرار تتحرك بنعومة (ألوان وخلفيات) خلال 0.2 ثانية.
- **:not(:disabled):active** → لحظة الضغط على زر غير معطّل: يتقلص 2% (scale 0.98) — إحساس لمسي بأن الزر "انضغط".

```
table tbody tr { transition: background-color 0.15s ease; }
table tbody tr:hover { background-color: #f5f7f9; }
table th:first-child { border-top-right-radius: var(--radius-sm); }
table th:last-child { border-top-left-radius: var(--radius-sm); }
```

- **tbody tr:hover** → أي صف في أي جدول بالمشروع يغمق قليلاً عند مرور الفأرة مع انتقال ناعم 0.15 ثانية — قاعدة واحدة تشمل كل الجداول (Write once).
- **th:first-child / :last-child** → تدوير الزاويتين العلويتين لرأس الجدول (اليمنى للأول واليسرى للأخير لأن الاتجاه RTL).

القسم 03: الخطوط (Typography)

```
.font-h1 { font-size: 24px; font-weight: 600; line-height: 32px; font-family: var(--font-ar); }
.font-h2 { font-size: 20px; font-weight: 600; line-height: 28px; font-family: var(--font-ar); }
.font-h3 { font-size: 18px; font-weight: 600; line-height: 28px; font-family: var(--font-ar); }
.font-body { font-size: 14px; font-weight: 400; line-height: 20px; font-family: var(--font-ar); }
.font-body-md { font-size: 14px; font-weight: 500; line-height: 20px; font-family: var(--font-ar); }
.font-small { font-size: 13px; font-weight: 400; line-height: 20px; font-family: var(--font-ar); }
.font-caption { font-size: 12px; font-weight: 400; line-height: 16px; font-family: var(--font-ar); }
.font-btn { font-size: 14px; font-weight: 500; line-height: 20px; font-family: var(--font-ar); }
.font-table-h { font-size: 13px; font-weight: 600; line-height: 16px; font-family: var(--font-ar); }
.font-table-b { font-size: 13px; font-weight: 400; line-height: 20px; font-family: var(--font-ar); }
.font-badge { font-size: 12px; font-weight: 500; line-height: 16px; font-family: var(--font-ar); }
.font-label-sm { font-size: 11px; font-weight: 500; line-height: 16px; font-family: var(--font-ar); }
.font-code-sm { font-size: 11px; font-weight: 500; line-height: 16px; font-family: var(--font-mono); }
.font-medium { font-weight: 500; }
.font-semibold { font-weight: 600; }
.font-bold { font-weight: 700; }
```

- **font-xxx.** → عائلة كلاسات طباعية موحدة (نظام تصميم Design System مصغّر): كل كلاس يحدد ثلاث قيم: حجم الخط font-size، سماكته font-weight (400 عادي، 500 متوسط، 600 شبه عريض، 700 عريض)، وارتفاع السطر line-height.
- **العلاقة بالصفحة** → العناوين h1/h2/h3 بأحجام 24/20/18، نص الجسم 14، الوصف الصغير 13، الشارات والتسميات 11-12، وخلايا الجداول 13.
- **line-height** → المسافة بين الأسطر؛ أكبر قليلاً من حجم الخط لقراءة عربية مريحة.
- **font-family: var(--font-ar)** → مكررة في كل كلاس لكنها تشير لمتغير واحد — التغيير من مكان واحد.
- **font-code-sm.** → الوحيد الذي يستخدم var(--font-mono) للأرقام الجامعية وعناوين IP.
- **آخر ثلاثة** → كلاسات وزن فقط (بدون حجم) تُدمج مع غيرها مثل font-bold + font-h1.

القسم 04: الحركات (Animations)

```
@keyframes slideDown {
  from { transform: translateY(-20px); opacity: 0; }
  to   { transform: translateY(0);    opacity: 1; }
}
@keyframes modalIn {
  from { opacity: 0; transform: translateY(14px) scale(0.98); }
  to   { opacity: 1; transform: translateY(0)    scale(1); }
}
```

- **@keyframes** → تعريف حركة (Animation) بإطارات (Frames): from نقطة البداية و to النهاية.
- **slideDown** → رسالة Toast تبدأ أعلى بـ 20px وشفافة (opacity:0) ثم تنزل لمكانها وتظهر — تستدعى من showToast في main.js بالاسم animation:slideDown.
- **modalIn** → النافذة المنبثقة تظهر وهي صغيرة 98% ومنزاحة 14px للأسفل ثم تكبر وترتقر في مكانها — تستدعى من قاعدة .fixed.inset-0 في قسم المكونات.
- **transform** → خاصية تحريك مكاني (تحريك/تكبير) أخف على المتصفح من تحريك margin.

القسم 05: التخطيط — السايدبار

```
.admin-sidebar {
  position: fixed; top: 0; bottom: 0; right: 0; width: 230px;
  background: var(--c-primary); color: #fff;
  display: flex; flex-direction: column; z-index: 100;
  transition: transform 0.3s cubic-bezier(0.16, 1, 0.3, 1);
}
```

- **position: fixed + top/bottom/right:0** → عمود ملصق بالحافة اليمنى للشاشة كاملاً ولا يتحرك مع التمرير.
- **width: 230px** → عرضه الثابت. **background: var(--c-primary)** → الكحلي من المتغير.
- **flex + flex-direction: column** → أبناؤه الثلاثة (شعار، قائمة، ذيل) مرصوصة عمودياً.
- **z-index:100** → فوق كل محتوى الصفحة وتحت الـ Toast والنافذة (z-50 في Tailwind = 50... في الحقيقة السايدبار 100 أعلى، لكنها لا تتعارض لأنهما لا يتقاطعان مكانيًا إلا مع الباك دروب z-index:90 الموضوع أقل منه عمداً ليغطي الشاشة ولا يغطي القائمة).
- **transition: transform 0.3s cubic-bezier(...)** → حركة انزلاق فتح/إغلاق على الجوال خلال 0.3 ثانية؛ cubic-bezier منحنى تسارع مخصص يعطي إحساس "نبض" ناعم في النهاية.

```
.sidebar-brand {
  display: flex; align-items: center; gap: 10px;
  padding: 16px 20px; border-bottom: 1px solid rgba(255,255,255,0.08);
}
.sidebar-brand img { width: 40px; height: 40px; object-fit: contain; background: transparent; padding: 0; flex-shrink: 0; }
.sidebar-brand .t1 { font-size: 14px; font-weight: 700; line-height: 1.3; }
.sidebar-brand .t2 { font-size: 11px; color: rgba(255,255,255,0.55); }
.sidebar-nav { flex: 1; overflow-y: auto; padding: 12px 10px; }
```

- **sidebar-brand** → الصورة والنص جنباً إلى جنب (flex) موسّطة عمودياً (align-items:center) بفراغ 10px، وخط سفلي شبه شفاف 8% أبيض يفصله عن القائمة.
- **img 40×40 + object-fit:contain** → الشعار يُحفظ بأبعاده داخل الإطار دون تشويه. flex-shrink:0 يمنع تصغيره.
- **.t1 / .t2** → اسم الجامعة أبيض عريض 14px، والوصف أبيض شفاف 55% بحجم 11px.
- **sidebar-nav** → flex:1 يأخذ كل المساحة المتبقية بين الشعار والذيل؛ إذا طالت القائمة يمرر داخلياً overflow-y:auto.

```
.side-item {
  display: flex; align-items: center; gap: 12px;
  width: 100%; padding: 11px 14px; margin-bottom: 2px;
  background: transparent; border: none; cursor: pointer;
  color: rgba(255,255,255,0.62); font-family: inherit;
  font-size: 14px; font-weight: 500; text-align: right;
  border-radius: 10px; transition: var(--tr);
}
.side-item:hover { background: rgba(255,255,255,0.08); color: #fff; }
.side-item.active { background: rgba(127,168,232,0.16); color: var(--c-accent-light); font-weight: 600; }
.side-item .material-symbols-outlined { font-size: 20px; }
.sidebar-foot { padding: 14px 20px; border-top: 1px solid rgba(255,255,255,0.08); font-size: 11px; color: rgba(255,255,255,0.4); }
```

- **side-item** → زر القائمة: أيقونة + نص بفراغ 12px، شفاف الخلفية، ونصه أبيض شفاف 62% (وضع "غير نشط").
- **font-family: inherit** → يرث خط الصفحة لأن الأزرار افتراضياً لا ترث الخط.
- **border-radius:10px + transition** → زوايا ناعمة وحركة لون عند hover.
- **hover:** → خلفية بيضاء شفافة 8% ونص أبيض كامل.

- **.active** → خلفية زرقاء شفافة 16% ونص أزرق فاتح (--c-accent-light) ووزن أثقل — هذه الحالة يضبطها navigateTo بنقل الكلاس.
- **الأيقونة 20px** → تكبير الأيقونة الافتراضية داخل أزرار القائمة.
- **sidebar-foot** → ذيل ثابت بخط علوي شفاف ونص باهت جداً 40%.

القسم 05 (تابع): الهيدر والمحتوى

```
.admin-main { margin-right: 230px; min-height: 100vh; display: flex; flex-direction: column; }
.admin-header {
  position: sticky; top: 0; z-index: 40;
  background: rgba(255,255,255,0.9); backdrop-filter: saturate(160%) blur(8px);
  border-bottom: 1px solid var(--c-border);
  padding: 12px 24px; display: flex; align-items: center; justify-content: space-between; gap: 12px;
}
.admin-header h1 { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.admin-header .h-right { display: flex; align-items: center; gap: 10px; }
```

- **admin-main** → هامش أيمن 230px = عرض السايدبار بالضبط حتى لا يغطيه؛ ارتفاع أدنى 100vh (شاشة كاملة) و flex عمودي (هيدر فوق محتوى).
- **admin-header** → sticky: يلصق بأعلى الشاشة عند التمرير (أذكى من fixed لأنه يحتفظ بمكانه الطبيعي في التدفق).
- **background شفاف 90% + backdrop-filter** → تأثير زجاجي (Glassmorphism): يضبب ويرفع تشبع ما خلف الشريط أثناء التمرير فوقه.
- **justify-between** → العنوان يميناً وأدوات المستخدم يساراً.
- **h1 القواعد الثلاث** → منع التفاف النص + إخفاء الفائض + "...": العنوان الطويل يظهر مقطوعاً بنقاط بدل كسر التصميم.
- **h-right** → مجموعة أدوات المستخدم بفراغ 10px بين عناصرها.

```
.hamburger {
  display: none; align-items: center; justify-content: center;
  width: 38px; height: 38px; border: none; border-radius: 10px;
  background: var(--c-surface-low); color: var(--c-primary); cursor: pointer;
}
.hamburger:hover { background: var(--c-surface-high); }
.sidebar-backdrop { display: none; position: fixed; inset: 0; background: rgba(9,20,38,0.45); z-index: 90; }
.sidebar-backdrop.active { display: block; }

main.admin-content { padding: 24px; width: 100%; max-width: 1160px; margin: 0 auto; flex: 1; }
```

- **hamburger** → مخفي display:none افتراضياً (لا حاجة له بجانب سايدبار دائم الظهور)؛ القاعدة في قسم التجاوب تظهره على الجوال. مربع 38×38 بحواف دائرية.
- **sidebar-backdrop** → الطبقة المعتمة: مخفية، وإن أُضيف كلاس active تصير block فوق كل شيء (z-index:90) تحت السايدبار (100). لونها كحلي شفاف 45%.
- **admin-content** → padding من كل الجهات 24px، أقصى عرض 1160px، و margin: 0 auto تعني "تمركز أفقي تلقائي" — على الشاشات العريضة جداً يبقى المحتوى في المنتصف.

```
.theme-toggle {
  width: 38px; height: 38px; border: none; border-radius: 10px;
  background: transparent; color: var(--c-text-secondary); cursor: pointer;
  display: inline-flex; align-items: center; justify-content: center;
  transition: var(--tr);
}
.theme-toggle:hover { background: var(--c-surface-container); }
.theme-toggle .material-symbols-outlined { font-size: 20px; }
.theme-toggle .tt-sun { display: none; }
```

- **theme-toggle** → زر الثيم: شفاف يغمق عند hover، والأيقونة 20px.
- **.tt-sun { display:none }** → أيقونة الشمس مخفية في الوضع النهاري (يظهر القمر فقط). في قسم الوضع الليلي تنعكس القاعدتان.

القسم 06: المكونات (Components)

```
.stat-card {
  background: #fff; border: 1px solid var(--c-border); border-radius: 12px;
  padding: 18px; display: flex; align-items: center; gap: 14px;
  transition: box-shadow 0.2s ease;
}
.stat-card:hover { box-shadow: 0 8px 20px -12px rgba(9,20,38,0.15); }
.stat-ico {
  width: 42px; height: 42px; border-radius: 10px; flex-shrink: 0;
  display: flex; align-items: center; justify-content: center;
}
.si-teal { background: #e3f4fc; color: var(--c-secondary); }
.si-green { background: #e2f6ec; color: var(--c-success); }
.si-neutral { background: #eef1f6; color: var(--c-text-secondary); }
.si-gold { background: var(--c-accent-bg); color: var(--c-accent); }
```

- **stat-card** → بطاقة إحصائية: بيضاء بإطار رمادي وزوايا 12px، أيقونتها يمين والنص يسارها (flex + gap 14px). عند hover يظهر ظل ناعم مرفوع — إحساس أنها "تفاعلية".
- **stat-ico** → مربع الأيقونة 42×42 داخلياً موسّط، و flex-shrink:0 يمنع انضغاطه.
- **si-teal / si-green / si-neutral / si-gold** → أربع ألوان جاهزة للمربع: خلفية فاتحة جداً + أيقونة بلون تشبع أعلى (الأزرق/الأخضر/الرمادي/الأزرق المميز). صف واحد لكل لون = لا تكرار.

```
.alert-card {
  display: flex; align-items: center; justify-content: space-between; gap: 12px; flex-wrap: wrap;
  background: #fff; border: 1px solid #f2d9d9; border-inline-start: 4px solid var(--c-error);
  border-radius: 12px; padding: 14px 16px;
}
.alert-card.ok { border-color: #cdeadd; border-inline-start-color: var(--c-success); }
```

- **alert-card** → بطاقة تنبيه: محتواها يمين وزرها يسار، وتلتف flex-wrap إذا ضاقت الشاشة.
- **border-inline-start: 4px** → شريط ملوّن سميك على "بداية الاتجاه" — وباستخدام inline بدل right تعمل صحيحاً في RTL و LTR معاً. الحمراء افتراضياً للتحذير.
- **.ok** → النسخة السليمة: نفس البطاقة لكن حدّها أخضر — الكلاس الإضافي فقط يغيّر اللونين.

```
.quick-action {
  display: flex; align-items: center; gap: 10px;
  background: #fff; border: 1px solid var(--c-border); border-radius: 12px;
  padding: 14px 16px; cursor: pointer; font-family: inherit;
  font-size: 14px; font-weight: 600; color: var(--c-primary);
  transition: var(--tr); text-align: right;
}
.quick-action:hover { border-color: var(--c-primary); transform: translateY(-1px); box-shadow: 0 8px 18px -12px
rgba(9,20,38,0.2); }
.quick-action .material-symbols-outlined { color: var(--c-secondary); }
```

- **quick-action** → زر الإجراء السريع: بطاقة بيضاء بنص كحلي شبه عريض.
- **:hover ثلاثية التفاعل** → إطار يصير كحلياً + البطاقة ترتفع (translateY(-1px + ظل أسفلها — حركة "طفو" كلاسيكية.
- **الأيقونة زرقاء** → قاعدة واحدة تلون كل أيقونات الأزرار السريعة.

```
.recent-row {
  display: flex; align-items: center; gap: 12px;
  padding: 10px 2px; border-bottom: 1px solid var(--c-border-light);
}
.recent-row:last-child { border-bottom: none; }
.recent-dot { width: 8px; height: 8px; border-radius: 50%; background: var(--c-accent); flex-shrink: 0; }
```

- **recent-row** → سطر نشاط في "آخر الأنشطة": نقطة + نص + وقت، بخط سفلي فاصل.
- **last-child:** → آخر سطر بلا خط سفلي حتى لا يوجد خط زائد تحت القائمة.
- **recent-dot** → نقطة دائرية 8px زرقاء مميزة — مؤشر بصري للنشاط.

```
.utab {
  background: transparent; border: none; cursor: pointer; font-family: inherit;
  font-size: 14px; font-weight: 500; color: var(--c-text-secondary);
  padding: 10px 4px; margin-left: 20px; border-bottom: 2px solid transparent;
  transition: var(--tr); white-space: nowrap;
}
.utab:hover { color: var(--c-primary); }
.utab.active { color: var(--c-primary); font-weight: 600; border-bottom-color: var(--c-accent); }

.panel-card { background: #fff; border: 1px solid var(--c-border); border-radius: 12px; padding: 18px; }
```

- **utab** → التبويب (من user-tab): نص رمادي بخط سفلي شفاف 2px؛ النشط يصير كحلياً عريضاً وخطه الأزرق المميز يظهر — شكل تبويب المتصفح.
- **margin-left:20px** → فراغ بين التبويبات (يسار في RTL يعني "بعد التبويب"). white-space:nowrap يمنع التفاف نصه.
- **panel-card** → أبسط مكون: لوحة بيضاء بإطار وزوايا 12px وحشوة 18px — تُستخدم لعشرات اللوحات في كل الشاشات.

```
.toggle-switch { position: relative; width: 44px; height: 24px; display: inline-block; }
.toggle-switch input { opacity: 0; width: 0; height: 0; }
.toggle-slider {
  position: absolute; cursor: pointer; inset: 0;
  background: var(--c-outline-variant); border-radius: 24px; transition: all 0.3s;
}
.toggle-slider::before {
  content: ''; position: absolute; height: 18px; width: 18px;
  right: 3px; bottom: 3px; background: #fff; border-radius: 50%; transition: all 0.3s;
}
.toggle-switch input:checked + .toggle-slider { background: var(--c-success); }
.toggle-switch input:checked + .toggle-slider::before { transform: translateX(-20px); }
.toggle-switch input:focus-visible + .toggle-slider { box-shadow: 0 0 0 3px rgba(0,164,114,0.25); }
```

- **مفتاح التفعيل (Toggle Switch)** → مبني بثلاثة عناصر: label الحاوية (44×24)، checkbox مخفي تماماً (opacity:0)، و span الشكل المرئي.
- **toggle-slider** → الكبسولة الرمادية. **before::** → دائرة بيضاء 18px على اليمين (RTL) — content:'' شرط لرسم عنصر وهمي (Pseudo-element).
- **input:checked + .toggle-slider** → **أهم سطر**: عند تفعيل checkbox، العلامة + تعني "العنصر التالي له مباشرة"، فيصبح لون الكبسولة أخضر.
- **checked::before + translateX(-20px):** → الدائرة تنزلق 20px لليسار — حركة القلبرة (Knob).
- **focus-visible:** → هالة خضراء عند التنقل بالكيبورد (إتاحة).
- **لا JavaScript هنا إطلاقاً** → كل الحركة CSS خالصة، و onchange في HTML فقط يرسل الإشعار.

```
.form-select { cursor: pointer; }
.form-input:focus, .form-select:focus, .form-textarea:focus,
input.h-11:focus, textarea:focus, select:focus {
  border-color: var(--c-secondary); box-shadow: 0 0 0 3px rgba(0,101,145,0.14); background-color: #ffffff;
}
```

- **حلقة تركيز الحقول** ← عند النقر في أي حقل: إطاره يصير أزرق + هالة (box-shadow منتشرة 3px زرقاء شفافة) + خلفيته تبيض — قاعدة واحدة تغطي كل الحقول بالمشروع حتى الحقول بلا كلاس form.
- **.form-select { cursor:pointer }** ← مؤشر اليد فوق القوائم المنسدلة.

```
.drop-zone { transition: border-color 0.2s ease, background 0.2s ease; }
.drop-zone:hover { background: #f7fbfe; }
.drop-zone.border-secondary { border-color: var(--c-secondary); background: #f0f8fc; }

.message-assistant { align-self: flex-start; border-top-right-radius: 4px; }
.message-user { align-self: flex-end; border-top-left-radius: 4px; }

.fixed.inset-0 > .bg-white { animation: modalIn 0.25s cubic-bezier(0.16,1,0.3,1) both; }
#toastContainer > div { box-shadow: 0 14px 34px -10px rgba(9,20,38,0.35); }
```

- **drop-zone** ← انتقال ناعم للإطار والخلفية؛ السطر الثالث: الكلاس border-secondary الذي يضيفه JavaScript أثناء السحب يلوّن المنطقة زرقاء.
- **رسائل المحادثة** ← message-assistant تلصق بالحافة اليمنى (بداية RTL) و message-user باليسرى؛ زاوية مدببة 4px عند منبع الرسالة تعطي شكل "فقاعة الدردشة".
- **.fixed.inset-0 > .bg-white.** ← أي نافذة مفتوحة (الطفل المباشر الأبيض) تدخل بحركة modalIn — القاعدة العامة تخدم كل النوافذ.
- **#toastContainer > div** ← ظل قوي لكل تنبيه Toast يُنشأ ديناميكياً.

القسم 07: أدوات مساعدة (Utilities)

هذه الكلاسات تُستخدم في HTML كأدوات جاهزة. بقية الأدوات (grid-cols و sm:/md:/lg: وغيرها) يولّدها Tailwind CDN المربوط في index.html — تم تدقيق هذا القسم وحُذف منه كل كلاس غير مستخدم في المشروع.

```
.shadow-sm { box-shadow: var(--shadow-sm); }
.shadow-lg { box-shadow: var(--shadow-lg); }

.mb-space-sm { margin-bottom: 8px; }
.mb-space-md { margin-bottom: 16px; }
.mb-space-lg { margin-bottom: 24px; }
.gap-space-xs { gap: 4px; }
.gap-space-sm { gap: 8px; }
.gap-space-md { gap: 16px; }
.gap-space-lg { gap: 24px; }
.py-space-sm { padding-top: 8px; padding-bottom: 8px; }
.py-space-md { padding-top: 16px; padding-bottom: 16px; }
```

- **shadow-sm / shadow-lg** ← ظلال جاهزة من المتغيرات.
- ***-mb-space** ← هوامش سفلية من نظام المسافات (Bottom Margin). ***-gap-space** ← فراغات شبكة Grid/Flex من نفس النظام. ***-py-space** ← حشوة علوية وسفلية معاً (Padding Y). كل قيمة تتطابق مع تعريف spacing في إعدادات Tailwind — توحيد كامل للمسافات.

```
.pe-10 { padding-left: 40px; }
.ps-11 { padding-right: 44px; }
```

- **ps / pe** ← اختصارات منطقية (Logical Properties): ps = padding-start أي بداية الاتجاه (اليمين في RTL) و pe = النهاية (اليسار). تستخدمان في حقول البحث لترك مكان لأيقونة البحث (44px = 11-ps من اليمين).

```
.px-2 { padding-left: 8px; padding-right: 8px; }
.px-2\.5 { padding-left: 10px; padding-right: 10px; }
.px-3 { padding-left: 12px; padding-right: 12px; }
.px-4 { padding-left: 16px; padding-right: 16px; }
.py-0\.5 { padding-top: 2px; padding-bottom: 2px; }
.py-1 { padding-top: 4px; padding-bottom: 4px; }
.py-1\.5 { padding-top: 6px; padding-bottom: 6px; }
.py-3 { padding-top: 12px; padding-bottom: 12px; }
.p-1\.5 { padding: 6px; }
.p-3 { padding: 12px; }
.p-5 { padding: 20px; }
.mt-1 { margin-top: 4px; }
.mb-1 { margin-bottom: 4px; }
.me-1 { margin-right: 4px; }
.me-2 { margin-right: 8px; }
.ms-auto { margin-right: auto; }
.ml-auto { margin-left: auto; }
```

- **نمط tailwind** ← الرقم = 4 × القيمة (px-3 = 12px). حرفا p/m للحشوة/الهامش، و x/y للاتجاهين الأفقي/العمودي، و s/e لبداية/نهاية الاتجاه (منطقية RTL-aware).
- **px-2\.5** ← الشرطة المائلة مع escaped نقطة (2\.5) أي قيمة نصفية 10px — صيغة Tailwind الشهيرة.
- **ms-auto / ml-auto** ← "ادفع العنصر لأقصى الجهة" بتمدد الهامش تلقائياً؛ تستخدمان لدفع زر تصدير CSV وزر الإرسال لطرف الصف.

```
.w-8 { width: 32px; } .h-8 { height: 32px; }
.w-full { width: 100%; }
.h-11 { height: 44px; }
.h-1\.5 { height: 6px; }
.max-w-lg { max-width: 32rem; }
```

- **المقاسات** ← عرض/ارتفاع ثابتة: h-11 (44px) هي الارتفاع الموحد لكل حقول الإدخال وأزرارها — مفتاح اتساق الواجهة. h-1.5 ارتفاع شريط التقدم.

```
.relative { position: relative; }
.absolute { position: absolute; }
.start-3\.5 { inset-inline-start: 14px; }
.top-1\/2 { top: 50%; }
.z-50 { z-index: 50; }
.inset-0 { inset: 0; }
```

- **المواقع (Positioning)** ← relative مرجع للأبناء المطلقين، absolute يخرج من التدفق ويتحرك نسبة لأقرب مرجع. start-3.5 و top-1/2 يستخدمان لأيقونة البحث داخل الحقل.
- **inset-inline-start** ← خاصية منطقية: "من جهة بداية الكتابة" — تتحول ليمين في RTL تلقائياً. inset:0 تعني "ملاصق من الجهات الأربع".

```
.text-start { text-align: right; }
.text-end { text-align: left; }
.text-center { text-align: center; }
```

- **محاذاة نص منطقية** ← start = يمين لأن الصفحة RTL (وليس left الثابت) — خلايا الجداول تلتزم بها.

```
.flex { display: flex; }
.grid { display: grid; }
.hidden { display: none; }
.flex-col { flex-direction: column; }
.flex-wrap { flex-wrap: wrap; }
.items-center { align-items: center; }
.items-start { align-items: flex-start; }
.justify-between { justify-content: space-between; }
.justify-center { justify-content: center; }
.flex-1 { flex: 1 1 0%; }
.gap-1 { gap: 4px; } .gap-2 { gap: 8px; } .gap-3 { gap: 12px; }
.space-y-1 > * + * { margin-top: 4px; }
.space-y-2 > * + * { margin-top: 8px; }
.space-y-3 > * + * { margin-top: 12px; }
.whitespace-nowrap { white-space: nowrap; }
```

- **flex/hidden** → أهم اثنين: hidden هو ما يخفي الشاشات والنوافذ. **flex-col** رصف عمودي، **flex-wrap** التفاف عند الضيق.
- **items-center** → توسيط عمودي للأبناء، **justify-between** توزيع بين الأطراف، **flex-1** "توسع واملأ المساحة المتبقية".
- **space-y-* > * + *** → **محدد ذكي**: "* + *" يعني "كل عنصر له أخ سابق" أي كل العناصر ما عدا الأول — فراغ عمودي بينها دون خط تحت الأول.

```
.rounded-lg { border-radius: var(--radius-sm); }
.rounded-xl { border-radius: var(--radius-md); }
.rounded-2xl { border-radius: var(--radius-lg); }
.rounded-full { border-radius: 9999px; }
.overflow-hidden { overflow: hidden; }
.overflow-x-auto { overflow-x: auto; }
.border-b { border-bottom-width: 1px; border-bottom-style: solid; }
.border-t { border-top-width: 1px; border-top-style: solid; }
.border-collapse { border-collapse: collapse; }
```

- **الزوايا** → أربع درجات من المتغيرات، و rounded-full (9999px) تجعل أي عنصر دائرة/كبسولة.
- **border-b/t** → خط سفلي/علوي بسماكة 1px (اللون يأتي من كلاس آخر مثل border-surface-container).

```
.bg-surface-container-low { background-color: var(--c-surface-low); }
.bg-surface-container-lowest { background-color: #ffffff; }
.bg-surface-container { background-color: var(--c-surface-container); }
.bg-surface-container-high { background-color: var(--c-surface-high); }
.bg-surface-container-highest { background-color: var(--c-surface-highest); }
.bg-primary { background-color: var(--c-primary); }
.bg-primary-container { background-color: var(--c-primary-container); }
.bg-secondary { background-color: var(--c-secondary); }
.bg-secondary-container { background-color: var(--c-secondary-container); }
.bg-secondary-fixed { background-color: var(--c-secondary-fixed); }
.bg-tertiary-container { background-color: var(--c-success-container); }
.bg-error { background-color: var(--c-error); }
.bg-error-container { background-color: var(--c-error-container); }
```

- **كلاسات الخلفيات** → ترجمة مباشرة للوحة الألوان: كل سطر يربط اسم كلاس بمتغير لون — تغيير اللون في :root ينتشر تلقائياً في كل الشاشات. tertiary-container يستخدم المتغير الأخضر الداكن (الترميز Material القديم حافظ عليه للتوافق).

```
.text-primary { color: var(--c-primary); }
.text-on-surface { color: var(--c-text); }
.text-on-surface-variant { color: var(--c-text-secondary); }
.text-on-primary { color: var(--c-on-primary); }
.text-on-secondary { color: #ffffff; }
.text-on-secondary-container { color: var(--c-on-secondary-container); }
.text-on-tertiary { color: #ffffff; }
.text-on-tertiary-container { color: var(--c-success); }
.text-on-error { color: #ffffff; }
.text-error { color: var(--c-error); }
.text-secondary { color: var(--c-secondary); }
.text-secondary-fixed { color: var(--c-secondary-fixed); }
.text-secondary-container { color: var(--c-secondary-container); }
.text-tertiary-fixed { color: var(--c-success-fixed); }
.text-outline { color: var(--c-outline); }
.text-outline-variant { color: var(--c-outline-variant); }
.text-on-primary-fixed { color: var(--c-secondary-fixed); }
```

- **كلاسات ألوان النص** ← نفس فلسفة الخلفيات. الأكثر استخداماً: text-primary (عناوين)، text-on-surface-variant (أوصاف رمادية)، text-error (تنبيهات)، text-outline (placeholder).

```
.transition-all { transition: var(--tr); }
.hover\:bg-surface-container:hover { background-color: var(--c-surface-container); }
.hover\:bg-surface-container-highest:hover { background-color: var(--c-surface-highest); }
.hover\:bg-primary:hover { background-color: var(--c-primary); }
.hover\:bg-primary-container:hover { background-color: var(--c-primary-container); }
.hover\:bg-error-container:hover { background-color: var(--c-error-container); }
.hover\:text-primary:hover { color: var(--c-primary); }
.hover\:underline:hover { text-decoration: underline; }
.focus\:outline-none:focus { outline: none; }

.text-\[10px\] { font-size: 10px; }
.text-\[18px\] { font-size: 18px; }
.text-\[20px\] { font-size: 20px; }
```

- **hover\: و focus\:** ← الشرطة المائلة المعاكسة (Backslash) تهرّب النقطتين في اسم الكلاس لأن : رمز خاص في CSS؛ في HTML تُكتب عادي مثل hover:bg-primary وتعمل فقط لحظة hover.
- **text-\[10px\].** ← صيغة القيمة الحرة (Arbitrary Value) في Tailwind: أي قيمة بين قوسين مربوطين؛ ه defining يدوياً هنا يضمن عملها حتى مع المحتوى الديناميكي.

القسم 08: الوضع الليلي (Dark Mode)

الآلية كلها تقوم على كلاس واحد: **body.dark-demo**. أي قاعدة تبدأ به تعمل فقط في الوضع الليلي. كلاس dark-demo يضيفه main.js عند التبديل ويحفظ الاختيار في localStorage.

```
body.dark-demo { background-color: #101418; color: #e2e2e5; color-scheme: dark; }
body.dark-demo ::selection { background: rgba(57,184,253,0.28); }
body.dark-demo .theme-toggle { color: #c3c7cd; }
body.dark-demo .theme-toggle:hover { background: #22262a; }
body.dark-demo .theme-toggle .tt-moon { display: none; }
body.dark-demo .theme-toggle .tt-sun { display: inline-block; }
```

- **الجسم** ← خلفية شبه سوداء ونص فاتح؛ color-scheme:dark يغير أيضاً عناصر المتصفح الأصلية (مثل حقل التاريخ).

- **تبديل أيقونات الثيم** ← القمر يختفي والشمس تظهر — عكس قاعدة الوضع النهاري بالضبط.

```
body.dark-demo .bg-white { background-color: #16191d; }
body.dark-demo .bg-surface-container-lowest { background-color: #191c1e; }
body.dark-demo .bg-surface-container-low { background-color: #1d2023; }
body.dark-demo .bg-surface-container { background-color: #22262a; }
body.dark-demo .bg-surface-container-high { background-color: #2a2e33; }
body.dark-demo .bg-surface-container-highest { background-color: #34383d; }
body.dark-demo .hover\:bg-surface-container:hover { background-color: #2a2e33; }
body.dark-demo .hover\:bg-surface-container-highest:hover { background-color: #3d4147; }
body.dark-demo .border-surface-container { border-color: #26292e; }
body.dark-demo .divide-y > * + * { border-top-color: #26292e; }
body.dark-demo .border-secondary { border-color: var(--c-secondary-container); }
```

- **الأسطح الليلية** ← نفس سلّم الدرجات لكن معكوس: كل خلفية فاتحة يقابلها لون داكن متدرج (16→34) — البطاقات أغمق قليلاً من الخلفية لتبقى مرئية.
- **الحدود** ← رمادي داكن #26292e بدل الفاتح، بما فيها فواصل الجداول divide-y.

```
body.dark-demo .text-primary { color: #e7eefc; }
body.dark-demo .hover\:text-primary:hover { color: #e7eefc; }
body.dark-demo .text-on-surface { color: #e2e2e5; }
body.dark-demo .text-on-surface-variant { color: #9aa0a6; }
body.dark-demo .text-outline { color: #9aa0a6; }
body.dark-demo .text-outline-variant { color: #4a4e55; }
body.dark-demo .text-secondary { color: var(--c-secondary-fixed-variant); }
body.dark-demo .text-error { color: #ffb4ab; }
body.dark-demo .text-on-tertiary-container { color: #4edea3; }
body.dark-demo .bg-secondary-fixed { background-color: #004c6e; }
body.dark-demo .text-on-secondary-fixed-variant { color: var(--c-secondary-fixed); }
body.dark-demo .text-on-primary-fixed { color: var(--c-secondary-fixed); }
```

- **قلب ألوان النص** ← كل النصوص الغامقة تصير فاتحة: العناوين أبيض مزرق، الثانوية رمادي فاتح، الأخطاء وردي فاتح، الأخضر يفتّح — قاعدة قراءة: "في الوضع الليلي نرفع إضاءة النص ونخفض تشبع الألوان قليلاً".

```
body.dark-demo .bg-secondary { background-color: #0c7db1; }
body.dark-demo .bg-primary { background-color: #16283f; box-shadow: inset 0 0 0 1px rgba(255,255,255,0.14); }
body.dark-demo .hover\:bg-primary:hover { background-color: #1b2f4a; }
body.dark-demo .hover\:bg-primary-container:hover { background-color: #223349; }
body.dark-demo .bg-error-container { background-color: #93000a; }
body.dark-demo .hover\:bg-error-container:hover { background-color: #a81c14; }
body.dark-demo input::placeholder, body.dark-demo textarea::placeholder { color: #8d9199; }
```

- **الأزرار ليلياً** ← درجات أفتح قليلاً من الليل حتى تُرى؛ الكحلي يصير أزرق داكن مع إطار داخلي شفاف (inset box-shadow) ليبرز عن الخلفية الداكنة.
- **placeholder** ← نص الحقول الإرشادي يفتّح ليبقى مقروءاً.

```
body.dark-demo button:focus-visible, ... { outline-color: var(--c-secondary-container); }
body.dark-demo .form-input:focus, ... {
  border-color: var(--c-secondary-container); box-shadow: 0 0 0 3px rgba(57,184,253,0.18); background-color:
#191c1e;
}
body.dark-demo .toggle-slider { background: #43474e; }

body.dark-demo .admin-header { background: rgba(16,20,24,0.85); border-bottom-color: #22262a; }
body.dark-demo .hamburger { background: #22262a; color: #e2e2e5; }
body.dark-demo .stat-card, body.dark-demo .panel-card, body.dark-demo .quick-action { background: #16191d; border-
color: #22262a; }
body.dark-demo .stat-card:hover { box-shadow: 0 8px 20px -12px rgba(0,0,0,0.6); }
body.dark-demo .quick-action { color: #e7eefc; }
body.dark-demo .quick-action:hover { border-color: rgba(57,184,253,0.55); ... }
body.dark-demo .si-teal { background: rgba(57,184,253,0.14); color: #7dd0fc; }
body.dark-demo .si-green { background: rgba(0,164,114,0.2); color: #4edea3; }
body.dark-demo .si-neutral { background: rgba(255,255,255,0.08); color: #c3c7cd; }
body.dark-demo .si-gold { background: rgba(127,168,232,0.16); color: var(--c-accent-light); }
body.dark-demo .alert-card { background: #16191d; border-color: rgba(255,180,171,0.3); }
body.dark-demo .alert-card.ok { border-color: rgba(110,231,183,0.3); }
body.dark-demo .recent-row { border-bottom-color: #22262a; }
body.dark-demo .recent-dot { background: #7fa8e8; }
body.dark-demo .utab { color: #9aa0a6; }
body.dark-demo .utab.active { color: #e7eefc; border-bottom-color: #7fa8e8; }
body.dark-demo table tbody tr:hover { background-color: #1d2024; }
body.dark-demo .drop-zone:hover { background: #1a2026; }
body.dark-demo .drop-zone.border-secondary { border-color: var(--c-secondary-container); background: #10222e; }
```

- **هذه الكتلة تعيد تلوين كل مكون** بنفس اسمه: الحقول عند التركيز تصير بخلفية داكنة وهالة زرقاء، البطاقات داكنة، مربعات الأيقونات si-* تصير ألواناً شفافة بدل خلفيات صلبة (تأثير أنعم على الداكن)، والتنبيهات حدود شفافة.
- **النقطة المهمة** ← لا يوجد "ملف ليلي منفصل": نفس الكلاسات تُعاد تعريفها تحت body.dark-demo — لذلك أي مكون جديد يُبنى بالكلاسات المشتركة يحصل على الوضع الليلي تلقائياً تقريباً.

القسم 09: التجاوب مع الشاشات (Responsive)

```
@media (max-width: 1023px) {
  .admin-sidebar { transform: translateX(100%); }
  .admin-sidebar.mobile-open { transform: translateX(0); box-shadow: -12px 0 32px rgba(0,0,0,0.25); }
  .admin-main { margin-right: 0; }
  .hamburger { display: inline-flex; }
}
```

- **@media (max-width: 1023px)** ← "طبّق هذه القواعد فقط عندما يكون عرض الشاشة 1023px أو أقل" — أي تحت حجم اللابتوب.
- **السايدبار يخرج من الشاشة** ← translateX(100%) يدفعه كاملاً لليمين خارج الرؤية (لأنه على اليمين أصلاً)؛ كلاس mobile-open الذي يضيفه main.js يعيده بانزلاق ناعم مع ظل كبير يوحي بالطبقة العائمة.
- **admin-main { margin-right: 0 }** ← المحتوى يتمدد لكل الشاشة لأن السايدبار لم يعد ظاهراً.
- **hamburger يظهر** ← inline-flex بدل none.

```
@media (max-width: 640px) {
  .admin-content { padding: 16px; }
  .admin-header { padding: 10px 16px; }
  table th, table td { padding-left: 10px; padding-right: 10px; }
}

@media (max-width: 520px) {
  .admin-header .h-right > span { display: none; }
  .admin-header .h-right { gap: 6px; }
}

@media (pointer: coarse) {
  button.p-1\.5 { min-width: 38px; min-height: 38px; }
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; }
}
```

- **640px** → تقليل الحشوات والهوامش على الجوال حتى تستغل الشاشة الضيقة، وتقليص حشوة خلايا الجداول.
- **520px** → إخفاء اسم المستخدم في الهيدر (span مباشر داخل h-right) مع بقاء الأفاتار والأزرار — الشاشات الضيقة جداً.
- **pointer: coarse** → "شاشة لمس" (بلا ماوس دقيق): أزرار أيقونات الجداول الصغيرة تُفرض بأبعاد لمس لا تقل عن 38px — معيار سهولة اللمس.
- **prefers-reduced-motion** → احترام إعداد المستخدم "تقليل الحركة" في نظامه: كل الحركات والانتقالات تصير لحظية — إتاحة (Accessibility) مهمة.

ملخص ملف css/style.css

- **وظيفة الملف:** المصدر الوحيد لكل تصميم المشروع: ألوان، خطوط، تخطيط، مكونات، وضع ليلي، وتجاوب.
- **أهم ما فيه:** متغيرات :root (مصدر الألوان الوحيد)، سلم الأسطح (surface tones)، قسم body.dark-demo الكامل، وقواعد RTL المنطقية (ps/pe/start/end).
- **علاقته بالملفات الأخرى:** يقرأ أسماء الكلاسات من index.html (ساكن) ومن template strings في app.js و main.js (ديناميكي) — لذلك أي كلاس جديد في JavaScript يجب تعريفه هنا.
- **أهم فكرة:** "كلاس واحد يخدم كل الشاشات" — hover لصف الجدول مرة واحدة يخدم 6 جداول، وقاعدة focus واحدة تخدم كل الحقول.

الملف 3: js/mock-data.js

Mock Data تعني "بيانات وهمية/تجريبية". هذا الملف هو **طبقة البيانات (Data Layer)**: كل بيانات المشروع في كائن واحد، ودوال قراءة صغيرة فوقه. عندما يُبنى Backend حقيقي، يُستبدل جسم هذه الدوال فقط بنداءات API ولا يتغير أي شيء آخر.

بداية الملف + الكائن الرئيسي

```
// ========================================
// js/mock-data.js
// جميع البيانات التجريبية مخزنة هنا
// لاحقاً API يسهل استبدالها بـ
// ========================================

const MOCK_DATA = {
```

- **// نص** → تعليق سطر واحد في JavaScript؛ المتصفح يتجاهله.
- **const MOCK_DATA = {** → إنشاء ثابت (Constant) اسمه MOCK_DATA — أي متغير لا يُعاد إسناده. قيمته كائن (Object) يبدأ بالقوس المجعّف ويحتوي كل بيانات المشروع. const يمنع الخطأ البرمجي من استبدال البيانات كلياً بالخطأ.

القسم الأول: إحصائيات لوحة التحكم

```
  // ===== إحصائيات لوحة التحكم =====
  dashboard: {
    conversationsToday: 1247,
    activeUsers: 389,
    totalDocuments: 56,
    unansweredPercentage: 12.5,
```

- **dashboard:** → خاصية (Property) داخل الكائن؛ اسمها يطابق ما تقرؤه getDashboardData().
- **conversationsToday: 1247** → زوج "مفتاح: قيمة" — عدد محادثات اليوم؛ يظهر في بطاقة statConversations بحركة العدّ.
- **activeUsers** → المستخدمون النشطون (بطاقة ثانية). **totalDocuments** → عدد ملفات قاعدة المعرفة (بطاقة ثالثة).
- **unansweredPercentage: 12.5** → نسبة الأسئلة غير المجابة؛ تظهر داخل بطاقة التنبيه الحمراء.

```
    weeklyUsage: {
      labels: ['السبت', 'الأحد', 'الإثنين', 'الثلاثاء', 'الأربعاء', 'الخميس', 'الجمعة'],
      values: [320, 450, 510, 480, 620, 710, 590]
    },
    monthlyUsage: {
      labels: ['الأسبوع 1', 'الأسبوع 2', 'الأسبوع 3', 'الأسبوع 4'],
      values: [2100, 2800, 3200, 3600]
    },
```

- **weeklyUsage** → كائن مصغر لبيانات الرسم البياني الأسبوعي.
- **labels** → مصفوفة (Array) أسماء المحاور الأفقية للرسم (أيام الأسبوع).
- **values** → مصفوفة أعداد المحادثات لكل يوم؛ طولها يطابق طول labels حتى يتطابق كل عمود مع يومه.
- **monthlyUsage** → نفس البنية للعرض الشهري (4 أسابيع) — زرا "أسبوع/شهر" في شاشة التقارير يبدل بينهما.
- **ملاحظة تصميمية** → هذه البيانات هي المصدر الوحيد للرسم البياني (كانت مكررة سابقاً في app.js وتم توحيدها هنا).

```
    frequentQuestions: [
      { question: 'ما أوقات الدوام الجامعي؟', count: 234 },
      ... (4 أسئلة أخرى بنفس البنية)
    ],
    failedQuestions: [
      { question: 'ما رسوم الالتحاق بالبرنامج الدولية؟', answers: [] },
      { question: 'هل يوجد مقرر عن الذكاء الاصطناعي؟', answers: ['لا يوجد مقرر حالياً'] },
      { question: 'كيف أحصل على منحة دراسية؟', answers: [] }
    ]
  },
```

- **frequentQuestions** → مصفوفة كائنات (Array of Objects)؛ كل عنصر = سؤال + عدد تكراره — يملأ جدول "أكثر الأسئلة تكراراً".
- **failedQuestions** → الأسئلة التي فشل الروبوت في الإجابة عنها.
- **answers: []** → مصفوفة فارغة تعني "بدون ردود" — وهي التي تحول لون السؤال أحمر وتغذي عدّاد التنبيه في الرئيسية (failedCount يعدّ العناصر التي مصفوفتها فارغة).
- **{** → إغلاق كائن dashboard، و **,** → فاصلة قبل الخاصية التالية.

القسم الثاني: المستندات

```
  // ===== المستندات =====
  documents: [
    { id: 1, name: 'منهج علم النفس الأساسي.pdf', type: 'PDF', size: '15.2 MB', date: '2026-09-15', status:
'indexed', category: 'كتب' },
    ... (9 مستندات أخرى بنفس البنية: محاضرات، أدلة، أبحاث، امتحانات...)
  ],
```

- **documents** → مصفوفة مستندات قاعدة المعرفة (10 عناصر) — يقرؤها renderDocuments().
- **id** → رقم فريد يمرر لـ confirmDelete عند الحذف.
- **type** → امتداد الملف يظهر داخل المربع الأزرق الصغير في الجدول.
- **size** → حجم الملف كنص جاهز للعرض ("15.2 MB").
- **status** → حالة الفهرسة: indexed مفهرس / processing قيد المعالجة / failed فشل — statusLabel تحولها لشارة ملوّنة.
- **category** → التصنيف؛ منه تُبنى خيارات قائمة الفلترة docCategory تلقائياً.

القسم الثالث: الطلاب

```
  // ===== الطلاب =====
  students: [
    { id: '2024001', name: 'أحمد محمد الشيخ', college: 'علوم الحاسوب', batch: '2024', lastActivity: '2026-09-20
14:30' },
    ... (11 طالباً آخراً بنفس البنية)
  ],
```

- **students** → مصفوفة 12 طالباً — يقرؤها renderStudents() ونموذج التعديل.
- **id نصي '2024001'** → الرقم الجامعي سلسلة نصية (String) وليس رقماً؛ حتى يحفظ الأصفار الأولى ويظهر بخط الأكواد في الجدول.
- **college / batch** → الكلية والدفعة؛ قيمتهما هي نفسها خيارات قوائم نموذج الإضافة/التعديل (studentFormHtml يعلّم selected على المطابق).
- **lastActivity** → آخر نشاط (تاريخ + ساعة) — بيانات إضافية مخزنة لا تعرضها الواجهة الحالية.

القسم الرابع: الخدمات الجامعية

```
  // ===== الخدمات الجامعية =====
  services: [
    { id: 1, name: 'الشؤون المالية', description: 'إدارة الشؤون المالية والموازنة الجامعية', office: 'مكتب الشؤون المالية', status: 'active', steps: ['تقديم الطلب', 'مراجعة المستندات', 'الموافقة', 'الإتمام'], links: ['https://finance.university.edu'] },
    ... (خدمتان أخريان: الرسوم، جول الامتحانات)
  ],
```

- **services** → 3 خدمات — يقرؤها renderServices() ويرسم كل واحدة بطاقة panel-card.
- **status: 'active'** → يحدد لون الشارة: أخضر "نشط" أو أحمر "غير نشط".
- **steps** → مصفوفة خطوات ترسم قائمة مرقمة <ol> داخل البطاقة.
- **links** → مصفوفة روابط تفتح بتبويب جديد ("target="_blank") بخط الأكواد.

القسم الخامس: المستخدمون والصلاحيات

```
  // ===== المستخدمون والصلاحيات =====
  users: [
    { id: 1, name: 'المسؤول العام', role: 'admin', active: true },
    ... (7 حسابات بأدوار admin / editor / viewer وحالات active مختلفة)
  ],
  permissions: {
    roles: ['admin', 'editor', 'viewer'],
    labels: ['مدير', 'محرر', 'مشاهد'],
    permissionKeys: [
      'manage_users', 'manage_content', 'manage_documents', 'view_reports',
      'manage_settings', 'view_logs', 'manage_chatbot', 'manage_students'
    ],
    permissionLabels: [
      'إدارة المستخدمين', 'إدارة المحتوى', 'إدارة المستندات', 'عرض التقارير',
      'إدارة الإعدادات', 'عرض السجلات', 'إدارة روبوت المحادثة', 'إدارة الطلاب'
    ],
    matrix: [
      [true, true, true, true, true, true, true, true],
      [false, true, true, true, false, true, true, false],
      [false, false, false, false, false, false, false, false]
    ]
  },
```

- **users** → حسابات الإدارة: لكل حساب دور role وحالة تفعيل active (تتحكم بمفتاح التبديل toggle-switch في الجدول).
- **permissions.roles** → الأدوار بأسمائها البرمجية (تُرسل لدالة updatePermission). **labels** → أسماؤها العربية المعروضة — فصل مقصود بين "اسم تقني" و"اسم معروض".
- **permissionKeys** → هوية كل صلاحية برمجياً، و **permissionLabels** → عناوين الأعمدة الثمانية المعروضة في المصفوفة.
- **matrix** → **مصفوفة ثنائية الأبعاد** (مصفوفة داخل مصفوفة): كل صف = دور، وكل خانة = هل يملك الدور الصلاحية ز؟ true/false تصير checkbox مفعّل/معطل. renderPermissions يمرر عليها بحلقتين متداخلتين.

القسم السادس: سجل النشاط

```
  // ===== سجلات النشاط =====
  activities: [
    { user: 'المسؤول العام', action: 'أضاف مستند جديد', time: '2026-09-20 15:30', ip: '192.168.1.10' },
    ... (11 سجلاً آخراً)
  ],
```

- **activities** → 12 سجل نشاط — يقرؤها: renderActivities (الجدول)، renderDashboard (آخر 5 أنشطة)، وقائمة فلتر المستخدمين (تُبنى من القيم الفريدة لخاصية user).
- **time** → نص "تاريخ ساعة"؛ الفلترة بالتاريخ تعمل بـ startsWith(dateFilter) أي مطابقة بداية النص مع التاريخ المختار.
- **ip** → عنوان IP يظهر بخط الأكواد في الجدول ويصدر في ملف CSV.

القسم السابع: الإعدادات

```
  // ===== إعدادات روبوت المحادثة =====
  chatbotSettings: {
    botName: 'مساعد الجامعة',
    welcomeMessage: 'مرحباً بك في مساعد الجامعة! كيف يمكنني مساعدتك اليوم؟',
    dialect: 'سعودي',
    systemPrompt: 'أنت مساعد جامعي ذكي يساعد الطلاب والموظفين...',
    temperature: 0.7,
    topK: 50,
    defaultResponse: 'عذراً، لم أتمكن من الإجابة على سؤالك...'
  },

  // ===== الإعدادات العامة =====
  generalSettings: {
    universityName: 'الجامعة الملكية',
    logo: null,
    language: 'ar',
    sessionDuration: 60,
    loginAttempts: 3,
    apiKey: 'sk-university-bot-2026-secret-key'
  }
};
```

- **chatbotSettings** → إعدادات الروبوت التي تقرأها شاشة المحادثة: botName و welcomeMessage يظهران في بداية المحادثة، و defaultResponse ضمن الردود العشوائية عند إرسال رسالة.
- **generalSettings** → إعدادات النظام التي يملؤها renderSettings() في حقول شاشة الإعدادات.
- **logo: null** → قيمة فارغة (Null) تعني "لا شعار مرفوع بعد".
- **};** → إغلاق الكائن الرئيسي MOCK_DATA نهائياً.

نهاية الملف: دوال القراءة (Data Accessors)

```
// ===== البيانات المجمعة =====
function getDashboardData() { return MOCK_DATA.dashboard; }
function getDocuments() { return MOCK_DATA.documents; }
function getStudents() { return MOCK_DATA.students; }
function getServices() { return MOCK_DATA.services; }
function getUsers() { return MOCK_DATA.users; }
function getPermissions() { return MOCK_DATA.permissions; }
function getActivities() { return MOCK_DATA.activities; }
function getChatbotSettings() { return MOCK_DATA.chatbotSettings; }
function getGeneralSettings() { return MOCK_DATA.generalSettings; }
```

- **function اسم() { return ... }** → دالة بسيطة تعيد (Return) جزءاً من البيانات — تسمى Accessor (دالة وصول).
- **لماذا لا نقرأ MOCK_DATA مباشرة من app.js؟** → فصل الطبقات: اليوم تعيد بيانات محلية، وغداً يُستبدل جسمها بـ fetch('/api/documents') — ولا يتغير أي سطر في app.js. هذه هي فكرة "يسهل استبدالها بـ API" في تعليق بداية الملف.
- **ملاحظة تاريخية** → كانت هذه الدوال مكررة نصياً في نهاية app.js أيضاً (تكرار حقيقي) وتم حذف النسخة الثانية — النسخ الوحيدة الآن هنا.

ملخص ملف js/mock-data.js

- **وظيفة الملف:** طبقة البيانات الوحيدة بالمشروع: كائن MOCK_DATA فيه 9 أقسام بيانات + 9 دوال قراءة فوقه.
- **أهم ما فيه:** مصفوفة matrix ثنائية الأبعاد للصلاحيات، بيانات الرسم البياني weekly/monthly، والفصل بين role و labels.
- **علاقته بالملفات الأخرى:** يُحمَّل أولاً في index.html؛ كل دوال app.js (و main.js عبر confirmDelete) تجلب منه عبر getXxx().
- **أهم ما تفهمه:** هو "قاعدة البيانات المؤقتة" — تغيير أي قيمة هنا يظهر فوراً في الواجهة دون لمس أي ملف آخر.

الملف 4: js/main.js

هذا الملف هو **الكود المشترك (Shared Code)**: كل وظيفة تحتاجها أكثر من شاشة تعيش هنا — قاعدة المشروع "Write once, reuse everywhere". يُحمَّل قبل app.js حتى تكون دواله جاهزة.

القسم 1: الوضع الليلي (Theme)

```
const THEME_KEY = 'app_theme';

function applyTheme(isDark, persist) {
  document.body.classList.toggle('dark-demo', isDark);
  if (persist) localStorage.setItem(THEME_KEY, isDark ? 'dark' : 'light');
  window.dispatchEvent(new CustomEvent('adminthemechange', { detail: { isDark } }));
}
```

- **const THEME_KEY** → ثابت يحمل اسم مفتاح التخزين؛ تعريفه مرة واحدة يمنع أخطاء الإملاء (لو كتبت المفتاح يدوياً في كل مكان واختلف حرف، انكسر الحفظ).
- **function applyTheme(isDark, persist)** → دالة تطبيق الثيم بمعاملين: isDark (هل ليلي؟) و persist (هل نحفظ الاختيار؟).
- **classList.toggle('dark-demo', isDark)** → toggle بمعامل ثانٍ يعني: أضف الكلاس إذا isDark=true وأزله إذا false — سطر واحد بدل if/else. هذا الكلاس هو ما يشغّل كل قسم body.dark-demo في style.css.
- **if (persist) localStorage.setItem(...)** → احفظ فقط عند تبديل المستخدم فعلياً (وليس عند فتح الصفحة). **? :** عامل شرطي مختصر: إن true خزّن 'dark' وإلا 'light'.
- **dispatchEvent(new CustomEvent('adminthemechange'))** → إطلاق حدث مخصص (Custom Event) عام في النافذة؛ الرسم البياني في app.js يسمعه ويعيد تلوين نفسه فوراً — آلية ربط بين الملفين بدون تبعية مباشرة.

```
function initTheme() {
  applyTheme(localStorage.getItem(THEME_KEY) === 'dark', false);
  document.querySelectorAll('#themeToggle, .theme-toggle').forEach((btn) => {
    btn.addEventListener('click', () => {
      applyTheme(!document.body.classList.contains('dark-demo'), true);
    });
  });
}
```

- **السطر الأول** → عند الإقلاع: اقرأ المحفوظ وقارنه بـ 'dark' (يعطي true/false) وطبّقه بدون حفظ (persist=false).
- **querySelectorAll('#themeToggle, .theme-toggle')** → ابحث عن زر الثيم بطريقتين (بالـ id و بالكلاس) — يجمع الاثنين احتياطاً.
- **forEach + addEventListener('click', ...)** → لكل زر: سجّل مستمع نقر (Event Listener) بدل onclick داخل HTML — الطريقة الحديثة الأنظف.
- **!contains('dark-demo')** → اقرأ الحالة الحالية واعكسها (علامة ! النفي): كان نهاراً؟ اجعله ليلاً — ثم احفظ (persist=true).

القسم 2: السايدبار (قائمة الجوال)

```
function initSidebar() {
  const toggle = document.getElementById('sidebarToggle');
  const sidebar = document.getElementById('adminSidebar');
  const backdrop = document.getElementById('sidebarBackdrop');
  if (!toggle || !sidebar || !backdrop) return;
  toggle.addEventListener('click', () => {
    sidebar.classList.add('mobile-open');
    backdrop.classList.add('active');
  });
  backdrop.addEventListener('click', closeSidebar);
}

function closeSidebar() {
  const sidebar = document.getElementById('adminSidebar');
  const backdrop = document.getElementById('sidebarBackdrop');
  if (sidebar) sidebar.classList.remove('mobile-open');
  if (backdrop) backdrop.classList.remove('active');
}
```

- **العناصر الثلاثة** → زر الهامبرغر (في الهيدر)، السايدبار نفسه، والطبقة المعتمة — كلها بمعرّفات موجودة في index.html.
- **if (!toggle || !sidebar || !backdrop) return;** → حماية (Guard Clause): إن لم يوجد أي عنصر اخرج فوراً؛ يمنع خطأ "null.nextSibling" لو استُخدم الملف بصفحة بلا سايدبار.
- **عند النقر على الهامبرغر** → أضف mobile-open للسايدبار (قاعدة @media تحرّكه داخل الشاشة بانزلاق) و active للمعتمة (تظهر).
- **عند النقر على المعتمة** → closeSidebar: تزيل الكلاسين فينزلق السايدبار ويختفي العتيم — استدعاء دالة باسمها كمرجع وليس بتنفيذ closeSidebar() فوري.
- **علاقة مخفية** → navigateTo في app.js تستدعي closeSidebar() أيضاً بعد كل تنقل، فيغلق القائمة تلقائياً على الجوال بعد اختيار شاشة.

القسم 3: التنبيهات (Toast)

```
function showToast(message, type = 'info') {
  const container = document.getElementById('toastContainer');
  const toast = document.createElement('div');
  const colors = {
    success: 'bg-tertiary-container text-on-tertiary',
    error: 'bg-error text-on-error',
    info: 'bg-secondary text-on-secondary',
    warning: 'bg-secondary-container text-on-secondary-container'
  };
  const icons = { success: 'check_circle', error: 'error', info: 'info', warning: 'warning' };
```

- **type = 'info'** → معامل بقيمة افتراضية (Default Parameter): لو استدعيت showToast('hi') فقط يعتبر type معلوماتية.
- **document.createElement('div')** → إنشاء عنصر div جديد في الذاكرة (لم يُضف للصفحة بعد).
- **colors** → قاموس (Object) يربط كل نوع بكلاسي لون خلفية + لون نص — نفس نظام ألوان style.css (أخضر النجاح، أحمر الخطأ، أزرق المعلومات، أزرق فاتح للتحذير).
- **icons** → قاموس مماثل لأسماء أيقونات Material لكل نوع.

```
  toast.className = `px-5 py-3 rounded-xl font-btn font-semibold ${colors[type] || colors.info} transition-all`;
  toast.style.cssText = 'animation:slideDown 0.3s ease;min-width:280px;';
  toast.innerHTML = `<span class="material-symbols-outlined text-[18px] me-2" style="vertical-
align:middle;">${icons[type] || icons.info}</span>${message}`;
  container.appendChild(toast);
```

- Backticks ` ` → Template Literal: نص يسمح بدمج المتغيرات بـ ${...} بدل جمع النصوص بـ +.
- **${colors[type] || colors.info}** → اختر لون النوع؛ || تعني "أو إن لم يوجد استخدم info" — حماية من نوع خاطئ.
- **style.cssText** → ضبط أنماط مباشرة لحظياً: حركة الدخول slideDown (المعرّفة في style.css) وحد أدنى للعرض.
- **innerHTML** → ضخ المحتوى داخل العنصر: أيقونة + نص الرسالة. (innerHTML يفسّر HTML بينما textContent تعرضه كنص مجرد).
- **appendChild(toast)** → اللحظة التي يدخل فيها التنبيه الشاشة فعلياً داخل الحاوية الموسطة أعلى الصفحة.

```
  setTimeout(() => {
    toast.style.opacity = '0';
    toast.style.transform = 'translateY(-10px)';
    setTimeout(() => toast.remove(), 300);
  }, 3000);
}
```

- **setTimeout(fn, 3000)** → نفّذ بعد 3 ثوانٍ: اجعل التنبيه شفافاً (opacity:0) وارفعه 10px — يبدو كأنه يتبخر للأعلى.
- **setTimeout الداخلي (300ms)** → انتظر انتهاء حركة الخفوت ثم **remove()** أي احذف العنصر من الصفحة كلياً حتى لا تتراكم عناصر مخفية في الذاكرة.
- **المعادلة الكاملة** → يظهر فوراً بحركة، يعيش 3 ثوانٍ، يختفي خلال 0.3 ثانية، ثم يُحذف.

القسم 4: النافذة المشتركة (Modal)

```
function openModal(title, bodyHtml, footerHtml) {
  document.getElementById('modalTitle').textContent = title;
  document.getElementById('modalBody').innerHTML = bodyHtml;
  document.getElementById('modalFooter').innerHTML = footerHtml || '';
  const overlay = document.getElementById('modalOverlay');
  overlay.classList.remove('hidden');
  overlay.classList.add('flex');
}

function closeModal() {
  const overlay = document.getElementById('modalOverlay');
  overlay.classList.add('hidden');
  overlay.classList.remove('flex');
}
```

- **openModal بثلاثة معاملات** → العنوان نصاً، جسم النافذة كـ HTML جاهز، وأزرار الأسفل.
- **textContent للعنوان** → نص مجرد (أأمن). **innerHTML للجسم** → لأنه يحتوي حقول أوسمة كاملة.
- **footerHtml || ''** → إن لم يُمرَّر أسفل ضع فراغاً — النافذة لا تنكسر أبداً بغيابه.
- **آلية الفتح** → أزل hidden (كان display:none) وأضف flex (يصبح حاوية موسّطة) — نفس ما تفعل قواعد style.css. الإغلاق بالعكس تماماً.
- **من يستدعيها؟** → إضافة طالب، تعديل طالب، تعديل خدمة، تأكيد حذف — كلها نفس النافذة بمحتوى مختلف. هذا هو معنى "نافذة مشتركة".

القسم 5: تأكيد الحذف

```
function confirmDelete(type, id, name) {
  openModal('تأكيد الحذف', `
    <div class="text-center">
      <span class="material-symbols-outlined text-[48px] text-error block mb-3">warning</span>
      <p class="font-body text-on-surface mb-1">هل أنت متأكد من حذف <strong>${name}</strong>؟</p>
      <p class="font-label-sm text-on-surface-variant">لا يمكن التراجع عن هذا الإجراء</p>
    </div>
  `, `
    <button class="... bg-surface-container-high ..." onclick="closeModal()">إلغاء</button>
    <button class="... bg-error ..." onclick="doDelete('${type}', '${id}', '${name}')">حذف</button>
  `);
}
```

- **confirmDelete(type, id, name)** → حوار حذف موحد: يستعمل من جدول المستندات ('type='document') وجدول الطلاب ('type='student).
- **لا يعرف شيئاً عن الحذف نفسه** → هو واجهة فقط؛ يبني جسم النافذة (أيقونة تحذير + اسم العنصر داخل strong) ويسلّم الأمر لـ openModal.
- **زر الحذف يرسل الثلاثية معه** → onclick="doDelete('${type}', '${id}', 'اسم')" — هذه الدالة في app.js (لأنها تعيد رسم الجداول — شاشة محددة) بينما الحوار هنا (مشترك). تقسيم مسؤوليات واضح.

القسم 6: ترقيم الصفحات (Pagination)

```
const paginationCallbacks = {};

function renderPagination(containerId, totalPages, currentPage, callback) {
  const container = document.getElementById(containerId);
  if (!container) return;
  paginationCallbacks[containerId] = callback;
  if (totalPages <= 1) { container.innerHTML = ''; return; }
```

- **paginationCallbacks** → كائن يعمل كـ "سجل هاتف": يحفظ لكل جدول الدالة التي يجب استدعاؤها عند تغيير صفحته (docPagination ← أعد رسم المستندات، وهكذا).
- **callback** → معامل من نوع دالة (Callback): renderPagination لا يعرف شيئاً عن المستندات أو الطلاب؛ يكفيه أن ينفذ الدالة الممررة — هذا ما يجعله مكوناً واحداً يخدم 3 جداول (سابقاً كان if/else مكرر لكل جدول).
- **totalPages <= 1** → صفحة واحدة؟ لا حاجة لأزرار ترقيم إطلاقاً — افرغ الحاوية واخرج.

```
  const navBtn = (icon, page, disabled) => `
    <button class="w-9 h-9 rounded-lg border ..." onclick="goPage('${containerId}', ${page})" ${disabled ?
'disabled style="opacity:0.3;cursor:not-allowed"' : ''}>
      <span class="material-symbols-outlined text-[18px]">${icon}</span>
    </button>`;
```

- **Arrow Function** → صيغة دالة مختصرة (المعاملات) => (النتيجة)؛ هنا "مصنع" يبني زر انتقال جاهزاً (سابق/تالي) — تعريف مرة واستخدام مرتين.
- **${disabled ? '...' : ''}** → إن كان الزر خارج النطاق أضف سمة disabled مع شفافية ومؤشر ممنوع، وإلا لا شيء.
- **goPage('${containerId}', ${page})** → لاحظ أن القيم تُطبع داخل النص: الزر الناتج في HTML يحمل نداءها مثبتاً بأسماء حقيقية.

```
  let html = navBtn('chevron_right', currentPage - 1, currentPage <= 1);
  for (let i = 1; i <= totalPages; i++) {
    const isActive = i === currentPage;
    html += `
      <button class="... ${isActive ? 'bg-primary text-on-primary border-primary' : 'bg-surface-container-lowest
... hover:bg-surface-container ...'}"
              onclick="goPage('${containerId}', ${i})">${i}</button>`;
  }
  html += navBtn('chevron_left', currentPage + 1, currentPage >= totalPages);

  container.innerHTML = html;
}

function goPage(containerId, page) {
  const callback = paginationCallbacks[containerId];
  if (callback) callback(page);
}
```

- **let html** → متغير نصي يتراكم فيه زر بعد زر؛ let تعني "سيُعاد إسناده" (عكس const).
- **chevron_right للسابق** → سهم لليمين = الرجوع لأن الترقيم RTL. **حلقة for** → زر لكل رقم صفحة من 1 إلى العدد.
- **${isActive ? ... : ...}** → الصفحة الحالية كحلي، والبقية بيضاء قابلة للنقر — نفس أسلوب تلوين زر أسبوع/شهر.
- **container.innerHTML = html** → ضخ كل الأزرار دفعة واحدة (أسرع من إضافة عنصر عنصر).
- **goPage** → جسر النقر: يقرأ الدالة المحفوظة للجدول المطلوب وينفذها برقم الصفحة الجديد؛ فتعيد renderDocuments مثلها إعادة رسم الصفحة الصحيحة.

القسم 7: عدّاد الأرقام المتحرك

```
function animateNumber(id, target) {
  const el = document.getElementById(id);
  if (!el) return;
  let current = 0;
  const increment = Math.max(1, Math.floor(target / 30));
  const timer = setInterval(() => {
    current += increment;
    if (current >= target) { current = target; clearInterval(timer); }
    el.textContent = current;
  }, 30);
}
```

- **animateNumber('statUsers', 389)** → يجعل الرقم "يطير" من 0 إلى قيمته بدل الظهور الفجائي.
- **Math.floor(target / 30)** → قسّم الهدف على 30 خطوة؛ **Math.max(1, ...)** → لكن لا تقل الخطوة عن 1 (لو الهدف 10 مثلاً تكون الخطوة 1).
- **setInterval(fn, 30)** → كرر كل 30 جزء من الألف من الثانية (~33 مرة/ثانية)؛ كل مرة زد الرقم الحالي.
- **if (current >= target)** → عند بلوغ الهدف ثبّته بالضبط وأوقف المؤقّت clearInterval — بدونها يستمر العدّ للأبد.
- **el.textContent = current** → اكتب القيمة الجديدة في البطاقة كل خطوة.

الإقلاع (Initialization)

```
document.addEventListener('DOMContentLoaded', () => {
  initTheme();
  initSidebar();
});
```

- **DOMContentLoaded** → حدث "اكتمل تحليل HTML"؛ تنفيذ التهيئة بعده يضمن وجود كل العناصر قبل البحث عنها (لأن الملف يُحمَّل آخر index.html فهو شبه مضمون، والحدث احتياط أصلح).
- **الجملة الكاملة** → عند جاهزية الصفحة: طبّق الثيم المحفوظ + اربط أزرار السايدبار. (أحداث app.js المستقلة تُسجَّل بنفس الأسلوب هناك).

ملخص ملف js/main.js

- **وظيفة الملف:** صندوق أدوات الواجهة المشترك: ثيم، سايدبار جوال، تنبيهات، نافذة واحدة، حوار حذف، ترقيم عام، وعدّاد أرقام.
- **أهم ما فيه:** نمط Callback في renderPagination (مكوّن واحد يخدم 3 جداول)، والحدث المخصص adminthemechange الذي يربط الثيم بالرسم البياني.
- **علاقته بالملفات الأخرى:** يقرأ معرّفات index.html، يضيف كلاسات معرّفة في style.css، وتستدعيه app.js (showToast/openModal/animateNumber) وتستدعي أنت دوال app.js منه (doDelete عبر onclick).
- **أهم ما تفهمه:** لا يحتوي أي بيانات أو شاشة — صرف "خدمات" للجميع؛ لذلك أي ميزة عامة جديدة مكانها هنا.

الملف 5: js/app.js

هذا الملف هو **عقل الشاشات**: لكل شاشة مجموعة دوال لها اسم واضح (renderDashboard, renderStudents...). هو يقرأ البيانات من mock-data.js، يستعين بالخدمات من main.js، ويكتب الناتج داخل عناصر index.html باستخدام كلاسات style.css.

الحالة العامة + الإقلاع

```
// ===== الحالة العامة =====
let usageChart = null;
let currentChartPeriod = 'week';
let docPage = 1, studentPage = 1, activityPage = 1;
const DOCS_PER_PAGE = 5, STUDENTS_PER_PAGE = 5, ACTIVITIES_PER_PAGE = 5;
```

- **let usageChart = null** → متغير يحفظ "كائن الرسم البياني" نفسه بعد إنشائه؛ null أي "لا شيء بعد". نحتاجه لنعرف هل يجب تدمير رسم قديم قبل رسم جديد.
- **currentChartPeriod** → المدة المعروضة حالياً (أسبوع افتراضياً) — تتذكرها عند إعادة رسم الرسم عند تبديل الثيم.
- **docPage = 1 ...** → رقم الصفحة الحالية لكل جدول من الجداول الثلاثة (تتغير عند التنقل بين الصفحات).
- **const DOCS_PER_PAGE** → ثوابت الحجم: 5 صفوف لكل صفحة في كل جدول؛ أسماء UPPERCASE عرف شائع للثوابت.

```
document.addEventListener('DOMContentLoaded', () => {
  renderDashboard();
  renderDocuments();
  renderStudents();
  renderUsers();
  renderPermissions();
  renderServices();
  renderReportTables();
  renderActivities();
  renderSettings();
  initChatMessages();
});
```

- **عند جاهزية الصفحة** → نفّذ دالة الرسم لكل شاشة مرة واحدة؛ الجداول الفارغة في index.html تمتلئ كلها فوراً حتى لو كانت الشاشة مخفية (المستخدم يجدها جاهزة عند التنقل).
- **الترتيب غير مهم هنا** → كل دالة مستقلة تمس عناصر مختلفة.

التنقل بين الشاشات

```
const PAGE_TITLES = {
  dashboard: 'الرئيسية',
  documents: 'قاعدة المعرفة',
  users: 'المستخدمون',
  chatbot: 'روبوت المحادثة',
  services: 'الخدمات الجامعية',
  reports: 'التقارير',
  settings: 'الإعدادات'
};

function navigateTo(page) {
  document.querySelectorAll('.side-item').forEach(item => {
    item.classList.toggle('active', item.dataset.page === page);
  });

  document.querySelectorAll('.page-section').forEach(section => {
    const show = section.id === 'page-' + page;
    section.classList.toggle('active', show);
    section.classList.toggle('hidden', !show);
  });

  const titleEl = document.getElementById('pageTitle');
  if (titleEl && PAGE_TITLES[page]) titleEl.textContent = PAGE_TITLES[page];

  closeSidebar();

  if (page === 'dashboard') renderDashboard();
  if (page === 'documents') renderDocuments();
  if (page === 'users') { renderStudents(); renderUsers(); renderPermissions(); }
  if (page === 'chatbot') initChatMessages();
  if (page === 'services') renderServices();
  if (page === 'reports') { renderReportTables(); renderChart(currentChartPeriod); renderActivities(); }
}
```

- **PAGE_TITLES** ← قاموس يربط الاسم التقني بالعنوان العربي الظاهر في الهيدر.
- **نفس اسم data-page = نفس مفتاح القاموس = نفس id القسم** ← هذه هي "اللغة المشتركة" بين السايدبار والأقسام.
- **السطر الأول داخل الدالة** ← لكل زر سايدبار: فعّل active فقط إذا سمة data-page عنده تطابق المطلوب — toggle بمعامل شرطي (true أضف/false أزل) يفعّل المطلوب ويزيله عن الباقي بسطر واحد.
- **السطر الثاني** ← لكل قسم شاشة: احسب show (هل id المطابق 'page-' + page؟) ثم أظهره أو أخفه. هذا جوهر نظام التنقل كله.
- **تحديث العنوان** ← && تعني "و": حدّث النص فقط إن وُجد العنوان والاسم في القاموس معاً — حماية من صفحة بلا عنوان.
- **closeSidebar()** ← دالة main.js: على الجوال تُغلق القائمة بعد الاختيار تلقائياً.
- **أسطر إعادة الرسم** ← كل تنقل يحدّث بيانات الشاشة (مهم للرسم البياني الذي يحتاج canvas ظاهراً ليرسم). شاشة users تستدعي ثلاث دوال لأن فيها تبويبين.

الشاشة الرئيسية (Dashboard)

```
function renderDashboard() {
  const d = getDashboardData();
  animateNumber('statConversations', d.conversationsToday);
  animateNumber('statUsers', d.activeUsers);
  animateNumber('statDocs', d.totalDocuments);
  renderDashboardAlert(d);
  renderRecentActivity();
}
```

- **renderDashboard** → منسّق صغير: يجلب البيانات من mock-data ثم يوزعها على الدوال المتخصصة بدل دالة عملاقة — نمط "دالة صغيرة لكل مهمة".
- **animateNumber** → من main.js: تحريك أرقام البطاقات الثلاث من صفر لقيمها.

```
function renderDashboardAlert(d) {
  const alertBox = document.getElementById('dashboardAlert');
  if (!alertBox) return;
  const failedCount = d.failedQuestions.filter(q => q.answers.length === 0).length;
  if (failedCount > 0) {
    alertBox.innerHTML = `... بطاقة alert-card "حمراء: عدد الأسئلة بدون إجابة + زر "مراجعة الآن
onclick="navigateTo('reports')" ...`;
  } else {
    alertBox.innerHTML = `... بطاقة alert-card ok "خضراء: "لا توجد تنبيهات ...`;
  }
}
```

- **filter(q => q.answers.length === 0)** → فلترة مصفوفة: أبقِ فقط الأسئلة التي مصفوفة إجاباتها فارغة؛ ثم .length يعطي عددها.
- **الشرط** → يوجد أسئلة بلا ردود؟ اعرض بطاقة حمراء فيها العدد والنسبة وزر ينقل للتقارير. وإلا بطاقة خضراء مطمئنة.
- **لاحظ** → نفس الحاوية dashboardAlert الفارغة في HTML، والمحتوى يتبدل كاملاً حسب البيانات.

```
function renderRecentActivity() {
  const list = document.getElementById('recentActivityList');
  if (!list) return;
  const recent = getActivities().slice(0, 5);
  list.innerHTML = recent.map(a => `
    <div class="recent-row">
      <span class="recent-dot"></span>
      <div class="flex-1">
        <div class="font-body-md text-on-surface">${a.user}</div>
        <div class="font-small text-on-surface-variant">${a.action}</div>
      </div>
      <span class="font-caption text-outline whitespace-nowrap">${a.time}</span>
    </div>
  `).join('');
}
```

- **slice(0, 5)** → خذ أول 5 عناصر فقط من سجل الأنشطة (slice لا تعدل الأصل بل تعيد نسخة).
- **map(a => ...)** → حوّل كل كائن نشاط إلى نص HTML لسطر recent-row؛ **join('')** → ادمج النصوص الناتجة في نص واحد بلا فواصل.
- **نمط map + join** → هذا هو النمط الأساسي لتوليد كل الجداول والقوائم في المشروع كله — تعلّمه وتعرف نصف app.js.

التقارير: الجداول + الرسم البياني

```
function renderReportTables() {
  const d = getDashboardData();

  const freqBody = document.getElementById('frequentQuestionsBody');
  if (freqBody) {
    freqBody.innerHTML = d.frequentQuestions.map((q, i) => `
      <tr class="transition-colors">
        <td class="py-3 px-3 text-on-surface-variant">${i + 1}</td>
        <td class="py-3 px-3 font-table-b text-on-surface">${q.question}</td>
        <td class="py-3 px-3"><span class="px-2.5 py-0.5 rounded-full font-label-sm bg-secondary-fixed
...">${q.count} تكرار</span></td>
      </tr>
    `).join('');
  }
  // مع حالة خاصة failedQuestionsBody ثم نفس الفكرة لجدول:
  // إن كانت المصفوفة فارغة يطبع صفاً واحداً "لا توجد أسئلة فاشلة" بأيقونة كبيرة
}
```

- **map((q, i) => ...)** ← المعامل الثاني i هو فهرس العنصر (0، 1، 2...)؛ نطبع i + 1 كرقم تسلسلي يبدأ من 1.
- **الشارة "N تكرار"** ← نفس كلاسات الشارات (كبسولة rounded-full بخلفية زرقاء فاتحة).
- **حالة الجدول الفارغ** ← بدل جدول أبيض محرج، تُطبع رسالة Empty State: أيقونة كبيرة + نص توضيحي — نفس النمط في كل الجداول.

```
function switchChartPeriod(period) {
  currentChartPeriod = period;
  renderChart(period);
  const activeClass = 'px-3 py-1 rounded-lg bg-primary text-on-primary font-btn';
  const idleClass = 'px-3 py-1 rounded-lg bg-surface-container text-on-surface-variant font-btn';
  document.getElementById('chartWeekBtn').className = period === 'week' ? activeClass : idleClass;
  document.getElementById('chartMonthBtn').className = period === 'month' ? activeClass : idleClass;
}
```

- **تذكّر المدة** في currentChartPeriod، ثم أعد رسم الرسم بها.
- **خدعة className الكامل** ← بدل إضافة/إزالة كلاسات فرعية، تُستبدل كاملة: الزر النشط يأخذ activeClass (كحلي) والآخر idleClass (رمادي) — أبسط طريقة لتبديل "شكلين" متكاملين.

```
function renderChart(period) {
  const canvas = document.getElementById('usageChart');
  if (!canvas || !window.Chart) return;
  const usage = getDashboardData();
  const data = period === 'week' ? usage.weeklyUsage : usage.monthlyUsage;

  const isDark = document.body.classList.contains('dark-demo');
  const lineColor = isDark ? '#39b8fd' : '#006591';
  const fillColor = isDark ? 'rgba(57, 184, 253, 0.12)' : 'rgba(0, 101, 145, 0.1)';
  const gridColor = isDark ? 'rgba(255, 255, 255, 0.08)' : 'rgba(0, 0, 0, 0.05)';
  const tickColor = isDark ? '#9aa0a6' : '#666666';

  if (usageChart) usageChart.destroy();
  usageChart = new Chart(canvas.getContext('2d'), {
    type: 'line',
    data: {
      labels: data.labels,
      datasets: [{
        label: 'عدد المحادثات',
        data: data.values,
        borderColor: lineColor,
        backgroundColor: fillColor,
        fill: true,
        tension: 0.4,
        pointBackgroundColor: lineColor,
        pointRadius: 5,
        pointHoverRadius: 7
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: { legend: { display: false } },
      scales: {
        y: { beginAtZero: true, grid: { color: gridColor }, ticks: { color: tickColor } },
        x: { grid: { display: false }, ticks: { color: tickColor } }
      }
    }
  });
}

window.addEventListener('adminthemechange', () => {
  if (usageChart) renderChart(currentChartPeriod);
});
```

- **if (!canvas || !window.Chart) return;** → حمايتان: لا عنصر، أو فشل تحميل مكتبة Chart.js (لا إنترنت)؟ اخرج بهدوء بدل انهيار الصفحة.
- **اختيار البيانات** → العامل الشرطي: أسبوع؟ خذ weeklyUsage وإلا monthlyUsage.
- **ألوان الرسم يدوية** → مكتبة Chart ترسم داخل canvas ولا تفهم كلاسات CSS؛ لذلك نقرأ حالة الثيم يدوياً ونمرر ألواناً صريحة (خط أزرق سماوي بالليل وأزرق داكن بالنهار، وخلفيات شفافة rgba).
- **if (usageChart) usageChart.destroy()** → قاعدة ذهبية في Chart: لا يصح رسم ثانٍ فوق الأول؛ دمّر القديم أولاً وإلا ظهرت طبقات فوق بعض.
- **new Chart(ctx, {...})** → إنشاء الرسم: getContext('2d') يعطي "فرشاة الرسم" على الـ canvas؛ type: line أي رسم خطي.
- **fill: true + tension: 0.4** → تعبئة ما تحت الخط بلون شفاف + انحناء ناعم للخط (0 = خطوط حادة).
- **responsive + maintainAspectRatio:false** → التزم بأبعاد الحاوية (الارتفاع 260px المحدد في HTML) ولا تحسب نسبة بنفسك.
- **legend مخفي** → لا داعي لمفتاح ألوان لأن المنحنى واحد.

- **المستمع الأخير** ← عند حدث تبديل الثيم القادم من main.js: أعد رسم الرسم بألوان الثيم الجديد — هذا هو "التعاون" بين الملفين.

قاعدة المعرفة: الجداول والرفع

```
function renderDocuments() {
  const docs = getDocuments();
  const searchEl = document.getElementById('docSearch');
  const search = searchEl ? searchEl.value.toLowerCase() : '';
  const catEl = document.getElementById('docCategory');

  if (catEl && catEl.options.length <= 1) {
    [...new Set(docs.map(d => d.category))].forEach(c => {
      const opt = document.createElement('option');
      opt.value = c; opt.textContent = c;
      catEl.appendChild(opt);
    });
  }
  const category = catEl ? catEl.value : '';
```

- **toLowerCase()** ← نص البحث بحروف صغيرة ليصبح البحث غير حساس لحالة الأحرف.
- **searchEl ? ... : ''** ← العامل الشرطي يمنع خطأ لو لم يوجد الحقل إطلاقاً.
- **options.length <= 1** ← املأ قائمة التصنيفات مرة واحدة فقط (خيار واحد = لم تُملأ بعد) — حتى لا تتكرر الخيارات عند كل إعادة رسم.
- **[...new Set(...)]** ← خدعة فريد القيم: map يستخرج التصنيفات، Set يحذف المكرر، و [...] تحول المجموعة لمصفوفة قابلة للتكرار بـ forEach.
- **createElement + appendChild** ← إنشاء خيار option وملؤه وإلحاقه بالقائمة (طريقة DOM الصريحة بدلاً من innerHTML).

```
  const filtered = docs.filter(d =>
    (!category || d.category === category) &&
    (d.name.toLowerCase().includes(search) || d.type.toLowerCase().includes(search) ||
d.category.toLowerCase().includes(search))
  );
  const totalPages = Math.ceil(filtered.length / DOCS_PER_PAGE);
  if (docPage > totalPages) docPage = 1;
  const start = (docPage - 1) * DOCS_PER_PAGE;
  const pageDocs = filtered.slice(start, start + DOCS_PER_PAGE);
```

- **سطر الفلترة** ← شرطان: (بلا تصنيف مختار أو التصنيف يطابق) **و** (الاسم أو النوع أو التصنيف يحوي نص البحث). includes تعني "يحتوي النص في أي موضع".
- **Math.ceil** ← تقريب لأعلى: 7 نتائج ÷ 5 لكل صفحة = صفحتان (1.4 تصير 2) — دائماً استعمل ceil في الترقيم.
- **if (docPage > totalPages) docPage = 1** ← إذا كانت على صفحة 3 وبقي بعد البحث صفحة واحدة، ارجع لأول صفحة (حماية من صفحة فارغة).
- **حساب القص** ← start = (الصفحة − 1) × الحجم؛ ثم slice يقطع الشريحة المطلوبة (صفحة 2: من العنصر 5 إلى 10).

```
  const body = document.getElementById('documentsBody');
  if (pageDocs.length === 0) {
    body.innerHTML = '<tr><td colspan="5">... أيقونة folder_off + "لا توجد مستندات" ...</td></tr>';
  } else {
    body.innerHTML = pageDocs.map(d => `
      <tr class="transition-colors">
        <td class="py-3 px-3">
          <div class="flex items-center gap-2">
            <div class="w-8 h-8 rounded-lg bg-secondary-container ... font-bold text-[10px]">${d.type}</div>
            <div>
              <div class="font-body text-on-surface">${d.name}</div>
              <div class="font-caption text-on-surface-variant">${d.category}</div>
            </div>
          </div>
        </td>
        <td ...>${d.type} · ${d.size}</td>
        <td ...>${d.date}</td>
        <td ...>${statusLabel(d.status)}</td>
        <td ... text-end>
          <div class="flex items-center justify-end gap-1">
            <button ... onclick="showToast('تم إعادة فهرسة ${d.name}','info')">refresh</button>
            <button ... onclick="showToast('عرض أجزاء ${d.name}','info')">visibility</button>
            <button ... onclick="confirmDelete('document', ${d.id}, '${d.name}')">delete</button>
          </div>
        </td>
      </tr>
    `).join('');
  }
  renderPagination('docPagination', totalPages, docPage, (p) => { docPage = p; renderDocuments(); });
}
```

- **"colspan="5** → خلية رسالة "لا توجد نتائج" تمتد على أعمدة الجدول الخمسة كلها.
- **خلية المستند** → مربع صغير يحمل نوع الملف + الاسم فوقه التصنيف — معلومتان في مساحة واحدة.
- **${statusLabel(d.status)}** → استدعاء دالة داخل Template — تُرجع شارة HTML جاهزة بلون الحالة.
- **أزرار الصف** → اثنان تجريبيان (يعرضان Toast) والثالث الحقيقي: confirmDelete من main.js بنوع document.
- **renderPagination ومعامل الدالة** → لاحظ (p) => { docPage = p; renderDocuments(); }: هذا هو الـ callback الذي يُخزَّن في main.js ويستدعيه goPage عند أي نقر — الدائرة تكتمل هنا.

```
function statusLabel(s) {
  const labels = { indexed: 'مفهرس', processing: 'قيد المعالجة', failed: 'فشل' };
  const colors = { indexed: 'bg-tertiary-container text-tertiary-fixed', processing: 'bg-secondary-container text-
on-secondary-container', failed: 'bg-error-container text-error' };
  return `<span class="px-2.5 py-0.5 rounded-full font-label-sm font-semibold whitespace-nowrap ${colors[s] ||
''}">${labels[s] || s}</span>`;
}

function filterDocuments() { docPage = 1; renderDocuments(); }
```

- **statusLabel** → مترجم صغير: حالة إنجليزية ← نص عربي + كلاسين للون؛ || s تعني "لو وصلت حالة جديدة غير معروفة اعرضها كما هي" — مرونة مستقبلية.
- **filterDocuments** → يرجع لصفحة 1 ثم يعيد الرسم — كل ضغطة حرف في البحث تستدعيه.

```
function handleDragOver(e) { e.preventDefault(); e.currentTarget.classList.add('border-secondary'); }
function handleDragLeave(e) { e.currentTarget.classList.remove('border-secondary'); }
function handleDrop(e) {
  e.preventDefault(); e.currentTarget.classList.remove('border-secondary');
  if (e.dataTransfer.files.length) simulateUpload(e.dataTransfer.files[0].name);
}
function handleFileSelect(e) { if (e.target.files.length) simulateUpload(e.target.files[0].name); }
```

- **e.preventDefault()** → ألغِ السلوك الافتراضي للمتصفح (كان سيفتح الملف بدل استقباله).
- **e.currentTarget** → العنصر الذي رُبط عليه الحدث (منطقة drop-zone)؛ نضيف/نزيل كلاس التلوين الأزرق.
- **e.dataTransfer.files** → الملفات المسحوبة؛ نأخذ الاسم الأول ونبدأ الرفع.
- **e.target.files** → نفس الفكرة لكن من حقل input العادي — الدالتان تلتحمان بنقطة واحدة: simulateUpload.

```
function simulateUpload(fileName) {
  const progressDiv = document.getElementById('uploadProgress');
  progressDiv.classList.remove('hidden');
  document.getElementById('uploadFileName').textContent = fileName;
  const fill = document.getElementById('uploadProgressFill');
  const percentEl = document.getElementById('uploadPercent');
  let percent = 0;
  const interval = setInterval(() => {
    percent += Math.random() * 15;
    if (percent >= 100) { percent = 100; clearInterval(interval); showToast('تم رفع وفهرسة الملف بنجاح',
'success'); setTimeout(() => progressDiv.classList.add('hidden'), 2000); }
    fill.style.width = percent + '%';
    percentEl.textContent = Math.round(percent) + '%';
  }, 300);
}
```

- **simulateUpload** → محاكاة رفع (لا يوجد خادم فعلياً): تُظهر شريط التقدم وتكتب اسم الملف.
- **Math.random() * 15** → زيادة عشوائية 0-15% كل 300ms — يعطي إيقافاً طبيعياً غير آلي.
- **عند 100%** → ثبّت الرقم، أوقف المؤقت، أظهر Toast نجاح، وبعد ثانيتين أخفِ الشريط.
- **fill.style.width = percent + '%'** → تمدد الشريط الأزرق بحسب النسبة (نفس الخاصية "style="width:0%" في HTML).
- **Math.round** → اعرض النسبة رقماً صحيحاً بلا كسور.

المستخدمون: الطلاب

```
function renderStudents() {
  const students = getStudents();
  const search = (document.getElementById('studentSearch')?.value || '').toLowerCase();
  const filtered = students.filter(s =>
    s.id.toLowerCase().includes(search) || s.name.toLowerCase().includes(search) ||
s.college.toLowerCase().includes(search)
  );
  // ... نفس معادلة الترقيم والقص المستخدمة في المستندات حرفياً ...
  // ثم map الدفعة، الكلية، الاسم بخط متوسط، الرقم بخط الأكواد: صفوفاً يبني
  // وزران: تعديل openEditStudentModal('${s.id}') وحذف confirmDelete('student', ...)
  renderPagination('studentPagination', totalPages, studentPage, (p) => { studentPage = p; renderStudents(); });
}

function filterStudents() { studentPage = 1; renderStudents(); }
```

- **? . القيمة الاختيارية** → studentSearch?.value تعني "إن وُجد العنصر خذ قيمته وإلا undefined" — ثم || '' يعطي نصاً فارغاً — سطر مدمج يحل ما تحتاجه if طويلة.
- **تكرار مقصود** → بنية renderStudents مطابقة لـ renderDocuments (بحث ← فلترة ← ترقيم ← رسم) — نمط موحد أسهل للفهم والصيانة.

```
function switchUserTab(tab) {
  const isStudents = tab === 'students';
  document.getElementById('panelStudents').classList.toggle('hidden', !isStudents);
  document.getElementById('panelAccounts').classList.toggle('hidden', isStudents);
  document.getElementById('tabStudents').classList.toggle('active', isStudents);
  document.getElementById('tabAccounts').classList.toggle('active', !isStudents);

  if (isStudents) renderStudents();
  else { renderUsers(); renderPermissions(); }
}

function goToStudents() {
  navigateTo('users');
  switchUserTab('students');
}
```

- **isStudents متغير منطقي (Boolean)** → متجره مرة ونستخدم عكسه !isStudents للعنصر المقابل — أربع جمل toggle متكاملة ومتناظرة.
- **الإعادة الرسومية** → كل تبديل يعيد رسم لوحة الهدف (بيانات مضمونة التحديث).
- **goToStudents** → دالة تركيبية يستدعيها زر "إضافة طالب" السريع في الرئيسية: افتح الشاشة ثم فعّل تبويب الطلاب — خطوتان مستخدموها كسلسلة واحدة.

```
// نموذج موحد لحقول الطالب (إضافة/تعديل)
function studentFormHtml(s = {}) {
  const collegeOptions = ['حقوق','تربية','الطب','الهندسة','علوم الحاسوب']
    .map(c => `<option ${s.college === c ? 'selected' : ''}>${c}</option>`).join('');
  const batchOptions = ['2023', '2024', '2025']
    .map(b => `<option ${s.batch === b ? 'selected' : ''}>${b}</option>`).join('');
  return `
    <div class="space-y-3">
      <div><label ...>الرقم الجامعي</label>
        <input ... id="studentIdInput" value="${s.id || ''}" placeholder="2026001 :مثال"></div>
      <div><label ...>الاسم الكامل</label>
        <input ... id="studentNameInput" value="${s.name || ''}" placeholder="أدخل الاسم"></div>
      <div class="grid grid-cols-2 gap-space-sm">
        <div><label>الكلية</label><select ... id="studentCollegeInput">${collegeOptions}</select></div>
        <div><label>الدفعة</label><select ... id="studentBatchInput">${batchOptions}</select></div>
      </div>
    </div>`;
}
```

- **s = {}** → قيمة افتراضية "كائن فارغ": استدعاء بلا بيانات = نموذج إضافة فارغ، واستدعاء بطالب = نموذج تعديل ممتلئ — دالة واحدة بدل نموذجين متكررين (كانا مكررين وقُدّما لدالة واحدة).
- **${s.college === c ? 'selected' : ''}** → علّم الخيار المطابق لبيانات الطالب المختاراً — إعادة بناء القائمة مع الاختيار الصحيح في كل مرة.
- **value="${s.id || ''}"** → قيمة الحقل؛ بمعامل فارغ تصير قيمة فارغة مع placeholder إرشادي.

```
function openAddStudentModal() {
  openModal('إضافة طالب جديد', studentFormHtml(), `
    <button ... onclick="closeModal()">إلغاء</button>
    <button ... onclick="addStudent()">إضافة</button>
  `);
}

function openEditStudentModal(id) {
  const s = getStudents().find(x => x.id === id);
  if (!s) return;
  openModal('تعديل بيانات الطالب', studentFormHtml(s), `
    ... onclick="saveStudent('${id}')">حفظ ...
  `);
}

function addStudent() {
  const id = document.getElementById('studentIdInput').value;
  const name = document.getElementById('studentNameInput').value;
  if (!id || !name) { showToast('الرجاء ملء جميع الحقول المطلوبة', 'error'); return; }
  showToast(`تم إضافة الطالب ${name} بنجاح`, 'success');
  closeModal();
  renderStudents();
}

function saveStudent(id) {
  const name = document.getElementById('studentNameInput').value;
  if (!name) { showToast('الرجاء ملء اسم الطالب', 'error'); return; }
  showToast(`تم تعديل بيانات ${name} بنجاح`, 'success');
  closeModal();
  renderStudents();
}
```

- **openAddStudentModal** → يملأ النافذة المشتركة (openModal من main.js): نموذج فارغ + زرا إلغاء/إضافة.
- **openEditStudentModal** → **find(x => x.id === id)** → ابحث في مصفوفة الطلاب عن أول عنصر يطابق المعرّف؛ إن لم يوجد اخرج (if (!s) return) — ثم نفس openModal لكن بالنموذج الممتلئ وزر حفظ يحمل id الطالب.
- **التحقق (Validation)** → !name || !id يعني "حقل فارغ؟" → Toast أحمر تحذيري + return يوقف الدالة فوراً — تحقق بسيط وواضح.
- **التسلسل بعد النجاح** → Toast أخضر ← أغلق النافذة ← أعد رسم الجدول — نمط ثابت تتكرره كل عمليات الحفظ في الملف.
- **ملاحظة صادقة** → لا يوجد هنا تعديل فعلي لمصفوفة MOCK_DATA (النسخة تجريبية للواجهة)؛ الرسائل تحاكي النتيجة النهائية حتى يُربط Backend.

المستخدمون: الحسابات والصلاحيات

```
function renderUsers() {
  const users = getUsers();
  const roleBadge = (role) => {
    const styles = { admin: 'bg-secondary-fixed text-on-primary-fixed', editor: 'bg-secondary-container text-on-secondary-container', viewer: 'bg-surface-container-high text-on-surface' };
    const labels = { admin: 'مدير', editor: 'محرر', viewer: 'مشاهد' };
    return `<span class="px-2.5 py-0.5 rounded-full font-label-sm ${styles[role]} font-medium">${labels[role]}</span>`;
  };
  const body = document.getElementById('usersBody');
  body.innerHTML = users.map(u => `
    <tr class="transition-colors">
      <td ...>${u.name}</td>
      <td ...>${roleBadge(u.role)}</td>
      <td ...><span class="... ${u.active ? 'bg-tertiary-container text-tertiary-fixed' : 'bg-error-container text-error'} ...">${u.active ? 'موقَف' : 'مفعّل'}</span></td>
      <td ...>
        <label class="toggle-switch inline-block">
          <input type="checkbox" ${u.active ? 'checked' : ''} onchange="toggleUser(${u.id}, this.checked)">
          <span class="toggle-slider"></span>
        </label>
      </td>
    </tr>
  `).join('');
}

function toggleUser(id, active) {
  showToast(`تم ${active ? 'تعطيل' : 'تفعيل'} المستخدم رقم ${id}`, active ? 'success' : 'warning');
}
```

- **roleBadge** ← دالة صغيرة داخلية (Arrow) تبني شارة الدور بلون خاص لكل دور — نفس فكرة statusLabel.
- **شارة الحالة** ← أخضر "مفعّل" أو أحمر "موقّف" حسب u.active بالعامل الشرطي.
- **مفتاح التفعيل** ← نفس بنية toggle-switch من HTML: checkbox مخفي + span الشكل. ${u.active ? 'checked' : ''} يجعل مفتاح حساب موقّف في وضع الإيقاف ابتداءً.
- **this.checked** ← داخل onchange تعني "حالة هذا المربع الآن بعد النقر" — تمرر true/false لدالة إشعار بسيطة تعرض رسالة نجاح/تحذير.

```
function renderPermissions() {
  const perm = getPermissions();
  document.getElementById('permHead').innerHTML = `
    <tr class="bg-surface-container text-on-surface-variant font-label-sm">
      <th class="py-3 px-3 text-start">الدور</th>
      ${perm.permissionLabels.map(l => `<th class="py-3 px-3 text-center font-semibold">${l}</th>`).join('')}
    </tr>
  `;
  document.getElementById('permBody').innerHTML = perm.roles.map((role, i) => `
    <tr class="transition-colors">
      <td ...>${perm.labels[i]}</td>
      ${perm.matrix[i].map((val, j) => `
        <td class="py-3 px-3 text-center"><input type="checkbox" ${val ? 'checked' : ''}
onchange="updatePermission('${role}', ${j}, this.checked)"></td>
      `).join('')}
    </tr>
  `).join('');
}

function updatePermission(role, permIndex, checked) {
  showToast(`${checked ? 'تم سحب' : 'تم منح'} صلاحية للمشرف ${role}`, 'info');
}
```

- **حلقة داخل حلقة (Nested Map)** → الرأس: صف واحد فيه عمود لكل صلاحية. الجسم: لكل دور صف، وداخل الصف خلية لكل قيمة في matrix[i] — هكذا تُرسم "مصفوفة" حقيقية من بيانات ثنائية الأبعاد.
- **perm.labels[i]** → الاسم العربي للدور بنفس فهرس الاسم التقني.
- **${val ? 'checked' : ''}** → true من البيانات يصير مربع إشارة مؤشّراً.
- **updatePermission** → إشعار فقط حالياً؛ الترسانة (الدور، رقم الصلاحية، الحالة) جاهزة لإرسالها للخادم لاحقاً.

الخدمات الجامعية

```
function renderServices() {
  const services = getServices();
  const grid = document.getElementById('servicesGrid');
  grid.innerHTML = services.map(s => `
    <div class="panel-card flex flex-col">
      <div class="flex items-start justify-between mb-space-sm">
        <h3 class="font-h3 text-primary">${s.name}</h3>
        <span class="... ${s.status === 'active' ? 'bg-tertiary-container text-tertiary-fixed' : 'bg-error-
container text-error'} ...">${s.status === 'active' ? 'غير نشط' : 'نشط'}</span>
      </div>
      <p class="font-body text-on-surface-variant mb-space-sm">${s.description}</p>
      <p class="font-small text-on-surface mb-space-sm"><span class="font-semibold">المكتب:</span> ${s.office}</p>
      <div class="mb-space-sm">
        <p class="font-small ... font-semibold">الخطوات:</p>
        <ol class="pr-4 space-y-1">${s.steps.map(st => `<li class="font-small text-on-surface">${st}
</li>`).join('')}</ol>
      </div>
      <div class="mb-space-sm">
        ${s.links.map(l => `<a href="${l}" target="_blank" class="font-code-sm text-secondary hover:underline
block">${l}</a>`).join('')}
      </div>
      <div class="mt-auto pt-space-md">
        <button ... onclick="openEditServiceModal(${s.id})">تعديل</button>
      </div>
    </div>
  `).join('');
}
```

- **بطاقة كاملة لكل خدمة** → بطاقة panel-card مرنة عمودياً (flex flex-col) تتكون من: عنوان + شارة الحالة، وصف، سطر المكتب، قائمة خطوات مرقمة ol، وروابط.
- **map متداخل للخطوات والروابط** → كل مصفوفة داخل البيانات تصير عناصر HTML بنداء map داخلي.
- **target="_blank"** → افتح الرابط بتبويب جديد. **pr-4** → إزاحة القائمة للداخل (يمين RTL).
- **mt-auto** → **تفصيلة ذكية**: دفع قسم زر التعديل لآخر البطاقة؛ حتى لو اختلفت أطوال الخدمات تبقى الأزرار مصطفّاة أسفل البطاقات.

```
function openEditServiceModal(id) {
  const s = getServices().find(x => x.id === id);
  if (!s) return;
  openModal('تعديل خدمة: ' + s.name, `
    <div class="space-y-3">
      <div><label ...>الاسم</label><input ... id="editServiceName" value="${s.name}"></div>
      <div><label ...>الوصف</label><textarea ... id="editServiceDesc" rows="3">${s.description}</textarea></div>
      <div><label ...>المكتب المسؤول</label><input ... id="editServiceOffice" value="${s.office}"></div>
      <div><label ...>الحالة</label><select ... id="editServiceStatus">active/inactive مع selected
للمطابق</select></div>
      <div><label ...>الخطوات</label><textarea ... id="editServiceSteps" rows="3">${s.steps.join('\n')}</textarea>
</div>
      <div><label ...>الروابط</label><textarea ... id="editServiceLinks" rows="3">${s.links.join('\n')}</textarea>
</div>
    </div>
  `, `حفظ يحمل/إلغاء زرا onclick="saveService(${s.id})" `);
}

function saveService(id) {
  const name = document.getElementById('editServiceName').value;
  if (!name) { showToast('الرجاء إدخال اسم الخدمة', 'error'); return; }
  showToast('تم حفظ تغييرات الخدمة بنجاح', 'success');
  closeModal();
  renderServices();
}
```

- **نموذج التعديل** ← كل خصائص الخدمة تصير حقول نصية؛ الحقول متعددة الأسطر (الخطوات/الروابط) تُسطّر بـ join('\n') — أي "اربطها بحرف سطر جديد" ليظهر كل عنصر بسطر.
- **نفس تسلسل الحفظ المعتاد** ← تحقق ← Toast ← إغلاق ← إعادة رسم.

روبوت المحادثة

```
function initChatMessages() {
  const container = document.getElementById('chatMessages');
  container.innerHTML = '';
  document.getElementById('chatBotNameDisplay').textContent = getChatbotSettings().botName;

  const initialMsg = document.createElement('div');
  initialMsg.className = 'message-assistant p-3 rounded-xl bg-secondary-container text-on-secondary-container max-
w-[80%] font-body';
  initialMsg.textContent = getChatbotSettings().welcomeMessage || 'مرحباً!';
  container.appendChild(initialMsg);
}

function appendChatMessage(className, text) {
  const container = document.getElementById('chatMessages');
  const msg = document.createElement('div');
  msg.className = className;
  msg.textContent = text;
  container.appendChild(msg);
  container.scrollTop = container.scrollHeight;
}
```

- **initChatMessages** ← تهيئة المحادثة كل فتح: صفّر الرسائل، اعرض اسم البوت من الإعدادات، ثم أضف رسالة الترحيب كأول فقاعة.
- **appendChatMessage** ← **دالة مساعدة استُخرجت لمنع التكرار**: إضافة فقاعة (كلاس + نص) تخدم رسائل المستخدم والبوت معاً — قبلها كان الكود مكرراً مرتين.

- **container.scrollTop = container.scrollHeight** → سطر "التمرير للأسفل": scrollHeight هو كامل ارتفاع المحتوى؛ وضعه في scrollTop يعني "انزل لآخر رسالة" — سلوك الدردشة المتوقع.
- **textContent هنا مقصود** → الرسائل نص مستخدم؛ textContent تمنع أي HTML خبيث من التنفيذ (أمان أساسي XSS).

```
function sendChat() {
  const input = document.getElementById('chatInput');
  const text = input.value.trim();
  if (!text) return;

  appendChatMessage('message-user p-3 rounded-xl bg-primary text-on-primary max-w-[80%] font-body ml-auto', text);
  input.value = '';

  setTimeout(() => {
    const responses = [
      'هذا سؤال ممتاز! سأحاول الإجابة بأفضل شكل ممكن.',
      'شكراً لسؤالك. هل لديك أي استفسارات أخرى؟',
      'لقد فهمت سؤالك. دعني أبحث في المعلومات المتاحة.',
      'يمكنني مساعدتك في هذا الشأن. يرجى الانتظار قليلاً.',
      getChatbotSettings().defaultResponse
    ];
    appendChatMessage('message-assistant p-3 rounded-xl bg-secondary-container text-on-secondary-container max-w-
[80%] font-body', responses[Math.floor(Math.random() * responses.length)]);
  }, 800 + Math.random() * 600);
}

function saveChatbotSettings() {
  showToast('تم حفظ إعدادات روبوت المحادثة بنجاح', 'success');
}
```

- **trim()** → احذف الفراغات من الطرفين؛ (!text) if يعني "رسالة فارغة أو مسافات فقط؟ لا شيء".
- **رسالة المستخدم** → فقاعة كحلية message-user مع ml-auto تدفعها لليسار (جهة المرسل في RTL).
- **input.value = ''** → أفرغ الحقل فور الإرسال كتطبيقات الدردشة.
- **الرد المؤجل** → setTimeout بمدة عشوائية (800-1400ms) يحاكي "التفكير"؛ ثم رد عشوائي: Math.floor(Math.random() * length) يعطي فهرساً صحيحاً بين 0 وطول المصفوفة − 1؛ ومن ضمنها defaultResponse من الإعدادات.
- **نقطة الربط المستقبلية** → استبدل هذا setTimeout بنداء API حقيقي عند جاهزية الخادم — بنية الواجهة جاهزة لذلك.

سجل النشاط

```
function renderActivities() {
  const filter = document.getElementById('activityUserFilter');
  if (filter && filter.options.length <= 1) {
    const uniqueUsers = [...new Set(getActivities().map(a => a.user))];
    uniqueUsers.forEach(u => {
      const opt = document.createElement('option');
      opt.value = u; opt.textContent = u;
      filter.appendChild(opt);
    });
  }

  let activities = getActivities();
  const dateFilter = document.getElementById('activityDateFilter')?.value || '';
  const userFilter = filter?.value || '';
  if (dateFilter) activities = activities.filter(a => a.time.startsWith(dateFilter));
  if (userFilter) activities = activities.filter(a => a.user === userFilter);

  // ... قص، صفحة: نفس معادلة الترقيم slice، رسم صفوف، renderPagination('activityPagination', ...) ...
}

function filterActivities() { activityPage = 1; renderActivities(); }
```

- **تعبئة الفلتر مرة واحدة** ← نفس حيلة options.length <= 1 و Set للقيم الفريدة المستخدمة في التصنيفات.
- **فلترة تراكمية** ← ابدأ بكل السجلات؛ فلتر التاريخ يطبق startsWith (النص يبدأ بالتاريخ المختار)، ثم فلتر المستخدم يطبق مطابقة تامة — الفلاتر تتضافر معاً.
- **activities = activities.filter(...)** ← إعادة إسناد النتيجة لنفس المتغير (let وليس const لهذا السبب).

```
function exportCSV() {
  const activities = getActivities();
  const rows = [['الوقت', 'الإجراء', 'المستخدم', 'IP']];
  activities.forEach(a => rows.push([a.user, a.action, a.time, a.ip]));
  const csv = rows.map(r => r.join(',')).join('\n');
  const blob = new Blob(['\ufeff' + csv], { type: 'text/csv;charset=utf-8;' });
  const link = document.createElement('a');
  link.href = URL.createObjectURL(blob);
  link.download = 'activity_log.csv';
  link.click();
  showToast('تم تصدير سجل النشاط بنجاح', 'success');
}
```

- **الصف الأول عناوين الأعمدة** ثم push لكل سجل كمصفوفة — مصفوفة مصفوفات = جدول.
- **join(',') لكل صف ثم join('\n') للصفوف** ← صيغة CSV: قيم بفواصل وأسطر بفواصل أسطر.
- **'\ufeff'** ← علامة BOM في بداية الملف؛ بدونها يفتح Excel النص العربي مشوهاً.
- **Blob + URL.createObjectURL** ← اصنع "ملفاً في الذاكرة" واعطه رابطاً مؤقتاً؛ ثم رابط وهمي بسمة download ونقر برمجي link.click() — ثلاثية تنزيل الملفات من المتصفح بدون خادم.

الإعدادات + تأكيد الحذف

```
function renderSettings() {
  const s = getGeneralSettings();
  document.getElementById('setUniName').value = s.universityName;
  document.getElementById('setApiKey').value = s.apiKey;
  document.getElementById('setSession').value = s.sessionDuration;
  document.getElementById('setAttempts').value = s.loginAttempts;
  document.getElementById('setLanguage').value = s.language;
}

function toggleApiKey() {
  const input = document.getElementById('setApiKey');
  const btn = input.nextElementSibling;
  const show = input.type === 'password';
  input.type = show ? 'text' : 'password';
  if (btn) btn.textContent = show ? 'إظهار' : 'إخفاء';
}

function saveSettings() {
  showToast('تم حفظ الإعدادات بنجاح', 'success');
}

function doDelete(type, id, name) {
  showToast(`تم حذف ${name} بنجاح`, 'success');
  closeModal();
  if (type === 'document') renderDocuments();
  if (type === 'student') renderStudents();
}
```

- **renderSettings** → ملء حقول شاشة الإعدادات من generalSettings — بدل قيم مكتوبة يدوياً في HTML؛ مصدر حقيقة واحد (البيانات).
- **toggleApiKey** → اقرأ النوع الحالي، اقلبه (password ↔ text)، وحدّث نص الزر المجاور nextElementSibling ليطابق ("إظهار" تصير "إخفاء").
- **doDelete** → الطرف الآخر لحوار confirmDelete في main.js: رسالة نجاح، إغلاق النافذة، ثم إعادة رسم الجدول المتأثر حسب type — لذلك هي في app.js (تعرف الشاشات) والحوار في main.js (عام).

ملخص ملف js/app.js

- **وظيفة الملف:** طبقة العرض (View Layer): 20+ دالة، كل مجموعة تخدم شاشة، وكلها تتبع نمطاً واحداً: اقرأ البيانات ← فلترها ← ارسم HTML داخل عناصر الصفحة ← اربط الأزرار.
- **أهم ما فيه:** دالة navigateTo (قلب التنقل)، نمط map + join لتوليد كل الجداول، renderChart و تفاعله مع الثيم، وسلاسل الحفظ الموحدة (تحقق ← Toast ← إغلاق ← رسم).
- **علاقته بالملفات الأخرى:** يقرأ من mock-data.js عبر getXxx()، يستدعي خدمات main.js (openModal/showToast/renderPagination/animateNumber/closeSidebar)، ويكتب في معرّفات index.html بكلاسات style.css.
- **أهم ما تفهمه:** لا يحتفظ بأي تصميم ولا بيانات — وسيط منظم؛ وكل دالة هنا قابلة للقراءة وحدها دون فهم البقية.

الخلاصة: كيف يعمل المشروع كاملاً

الأدوار الثلاثة

- **HTML (index.html)** → بناء الصفحة: الهيكل والعناصر والمعرّفات. لا يحمل بيانات حقيقية — جداوله فارغة عمداً.
- **CSS (style.css)** → تصميم الصفحة: كل شكل ولون وحركة. يعمل بالكلاسات فقط، ويتفاعل مع الحالة (dark-demo, active, hidden, mobile-open) بتغيير المظهر لا المحتوى.
- **JavaScript (main.js + app.js + mock-data.js)** → التفاعل والحركة والوظائف: يقرأ البيانات، يبني HTML ديناميكياً، يسمع النقرات، ويبدّل الكلاسات.

رحلة طلب واحد من البداية للنهاية (مثال: حذف مستند)

1) المستخدم ينقر أيقونة الحذف في جدول المستندات — زر ولّده app.js داخل documentsBody في index.html.

2) onclick يستدعي confirmDelete('document', 3, 'اسم') في main.js.

3) confirmDelete يستدعي openModal (نفسها main.js) فتُملأ نافذة index.html المشتركة ويُستبدل hidden بـ flex.

4) style.css تشعر بالتغيير: قاعدة .bg-white. < fixed.inset-0 تشغّل حركة modalIn، والشاشة تستلم الألوان الليلية إن كان dark-demo فعّالاً.

5) المستخدم يؤكد ← doDelete في app.js: showToast (main.js تصنع عنصراً في toastContainer وتحركه slideDown) ← ← closeModal renderDocuments تعيد رسم الجدول ببيانات mock-data.js.

خريطة التعاون بين الملفات

- **mock-data.js → app.js:** بيانات عبر دوال getXxx() (نقطة استبدال API المستقبلية).
- **app.js → main.js:** خدمات عامة: showToast, openModal/closeModal, renderPagination/goPage, animateNumber, closeSidebar.
- **index.html → app.js:** كتابة HTML ديناميكي داخل المعرّفات + أحداث onclick تعيد استدعاء دواله ودوال main.js.
- **style.css → الجميع:** تشكل كل كلاس (الساكن والديناميكي)، وتتغير سلوكياً مع كلاسات الحالة التي يبدّلها JavaScript.
- **الحدث المخصص adminthemechange:** قناة اتصال من main.js (الثيم) إلى app.js (الرسم البياني) — مثال على فصل الاهتمامات (Separation of Concerns).

المبادئ التي بُني عليها المشروع

- **Write once, reuse everywhere** → نافذة واحدة لكل النوافذ، مكون ترقيم واحد لكل الجداول، toast واحد لكل الرسائل، style.css واحد لكل الشاشات.
- **الفصل بين الطبقات** → بيانات (mock-data) / خدمات عامة (main) / شاشات (app) / شكل (css) / هيكل (html).
- **دوال صغيرة وواضحة** → كل دالة تفعل شيئاً واحداً باسم يصفه (renderDashboardAlert, filterStudents...).
- **جاهزية الترقية** → استبدال دوال getXxx بنداءات API هو كل ما يلزم لتحويل المشروع من تجريبي إلى حقيقي دون لمس الواجهة.

— نهاية الشرح —